# REVISÃO: Tipos de Subrotinas

- procedimentos

```
void ImprimeSoma (int a, int b)
{
    printf("%d", a+b);
}
```

- funções

```
int Soma (int a, int b)
{
    return (a+b);
}
```

# Parâmetros de uma subrotina em C Exemplo

Chamada por valor: Qualquer alteração feita no parâmetro não reflete em alteração no argumento.

Chamada por referência: Todas as alterações realizadas no parâmetro, refletem em alterações no argumento, ou seja, ambas as variáveis apontam para o mesmo endereço de memória. Para isso, é necessário que seja passado o endereço do argumento e o parâmetro receba-o na forma de ponteiro.

# Definição

Variável

É um espaço reservado de memória usado para guardar um ***valor*** que pode ser modificado pelo programa;

Ponteiro

É um espaço reservado de memória usado para guardar o ***endereço de memória*** de uma outra variável.

Um ponteiro é uma variável como qualquer outra do programa – sua diferença é que ela não armazena um valor inteiro, real, caractere ou booleano.

Ela serve para armazenar endereços de memória (são valores inteiros sem sinal).

# Declaração

Como qualquer variável, um ponteiro também possui um tipo

```
//declaração de variável
tipo_variável  nome_variável;

//declaração de ponteiro
tipo_ponteiro *nome_ponteiro;
```

É o ***asterisco*** (*) que informa ao compilador que aquela variável não vai guardar um valor mas sim um endereço para o tipo especificado.

```
int x;
float y;
struct ponto p;

int *x;
float *y;
struct ponto *p;
```

# Declaração

Exemplos de declaração de variáveis e ponteiros

```
int main(){
    //Declara um ponteiro para int
    int *p;
    //Declara um ponteiro para float
    float *x;
    //Declara um ponteiro para char
    char *y;
    //Declara um ponteiro para struct ponto
    struct ponto *p;
    //Declara uma variável do tipo int e um ponteiro para int
    int soma, *p2,;

    return 0;
}
```

# Declaração

Na linguagem C, quando declaramos um ponteiro nós informamos ao compilador para que tipo de variável vamos apontá-lo.

Um ponteiro **int*** aponta para um inteiro, isto é, **int**

Esse ponteiro guarda o endereço de memória onde se encontra armazenada uma variável do tipo **int**

# Inicialização

Ponteiros apontam para uma posição de memória.

**Cuidado**: Ponteiros não inicializados apontam para um lugar indefinido.

Exemplo

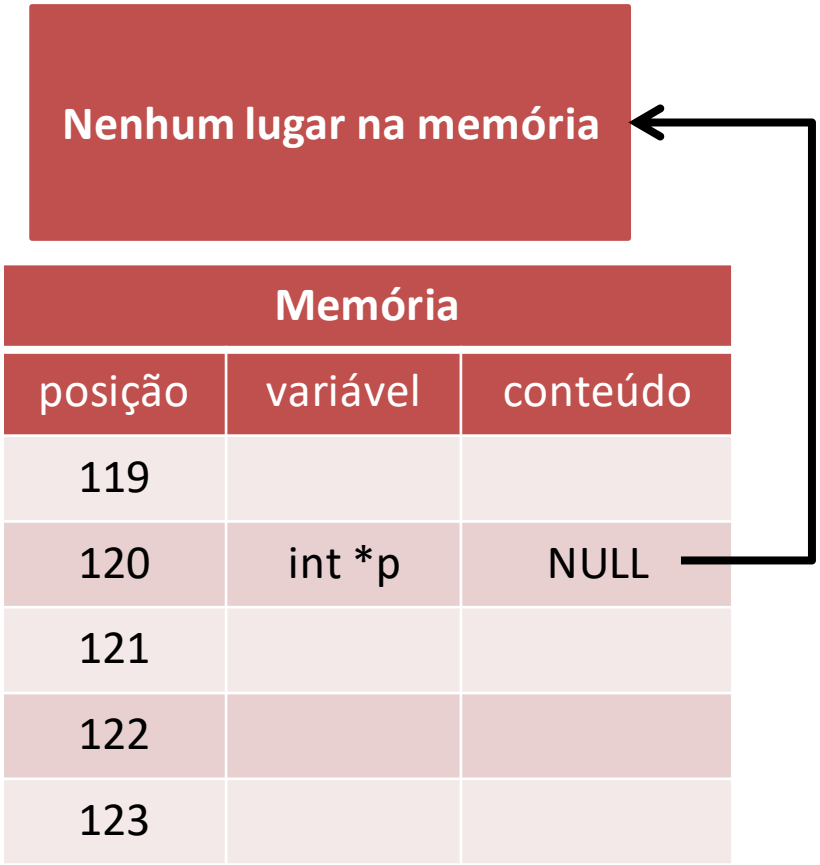
```
int *p;
```

| Memória | | |
|---|---|---|
| posição | variável | conteúdo |
| 119 | | |
| 120 | int *p | ???? |
| 121 | | |
| 122 | | |
| 123 | | |

# Inicialização

Um ponteiro pode ter o valor especial NULL que é o endereço de nenhum lugar.
Exemplo

```
int *p = NULL;
```



# Inicialização

Os ponteiros devem ser inicializados antes de serem usados.
Assim, devemos apontar um ponteiro para um lugar conhecido
Podemos apontá-lo para uma variável que já exista no programa.

| Memória | | |
|---|---|---|
| posição | variável | conteúdo |
| 119 | | |
| 120 | int *p | 122 |
| 121 | | |
| 122 | int c | 10 |
| 123 | | |

# Inicialização

O ponteiro armazena o endereço da variável para onde ele aponta.

Para saber o endereço de memória de uma variável do nosso programa, usamos o operador **&**.

Ao armazenar o endereço, o ponteiro estará apontando para aquela variável

| Memória | | |
|---|---|---|
| posição | variável | conteúdo |
| 119 | | |
| 120 | int *p | 122 |
| 121 | | |
| 122 | int c | 10 |
| 123 | | |

```
int main(){
    //Declara uma variável int contendo o valor 10
    int c = 10;
    //Declara um ponteiro para int
    int *p;
    //Atribui ao ponteiro o endereço da variável int
    p = &c ;

    return 0;
}
```

# Utilização

Tendo um ponteiro armazenado um endereço de memória, como saber o valor guardado dentro dessa posição?

# Utilização

Para acessar o **valor** guardado dentro de uma posição na memória apontada por um ponteiro, basta usar o operador ***asterisco*** "*" na frente do nome do ponteiro

```
int main(){
    //Declara uma variável int contendo o valor 10
    int c = 10;
    //Declara um ponteiro para int
    int *p;
    //Atribui ao ponteiro o endereço da variável int
    p = &c;
    printf("Conteudo apontado por p: %d \n",*p);// 10
    //Atribui um novo valor à posição de memória apontada por p
    *p = 12;
    printf("Conteudo apontado por p: %d \n",*p);// 12
    printf("Conteudo de count: %d \n",c);// 12

    return 0;
}
```

# Utilização

***p** :conteúdo da posição de memória apontado por **p**;
**&c**: o endereço na memória onde está armazenada a variável **c**.

```
int main(){
    //Declara uma variável int contendo o valor 10
    int c = 10;
    //Declara um ponteiro para int
    int *p;
    //Atribui ao ponteiro o endereço da variável int
    p = &c;
    printf("Conteudo apontado por p: %d \n",*p);// 10
    //Atribui um novo valor à posição de memória apontada por p
    *p = 12;
    printf("Conteudo apontado por p: %d \n",*p);// 12
    printf("Conteudo de count: %d \n",c);// 12

    return 0;
}
```

# Utilização

De modo geral, um ponteiro só pode receber o endereço de memória de uma variável do mesmo tipo do ponteiro

- Isso ocorre porque diferentes tipos de variáveis ocupam espaços de memória de tamanhos diferentes
- Na verdade, nós podemos atribuir a um ponteiro de inteiro (**int ***) o endereço de uma variável do tipo **float**. No entanto, o compilador assume que qualquer endereço que esse ponteiro armazene obrigatoriamente apontará para uma variável do tipo **int**
- Isso gera problemas na interpretação dos valores

# Utilização

```
int main(){
    int *p, *p1, x = 10;
    float y = 20.0;
    p = &x;
    printf("Conteudo apontado por p: %d \n",*p);

    p1 = p;
    printf("Conteudo apontado por p1: %d \n",*p1);

    p = &y;
    printf("Conteudo apontado por p: %d \n",*p);
    printf("Conteudo apontado por p: %f \n",*p);
    printf("Conteudo apontado por p: %f \n",*((float*)p));

    return 0;
}
```

```
Conteudo apontado por p: 10
Conteudo apontado por p1: 10
Conteudo apontado por p: 1101004800
Conteudo apontado por p: 0.000000
Conteudo apontado por p: 20.000000
```

# Operações com ponteiros

Atribuição

p1 aponta para o mesmo lugar que p2;

```
int *p, *p1;
int c = 10;
p = &c;
p1 = p;//equivale a p1 = &c;
```

a variável apontada por p1 recebe o mesmo conteúdo da variável apontada por p2;

```
int *p, *p1;
int c = 10, d = 20;
p = &c;
p1 = &d;

*p1 = *p;//equivale a d = c;
```

# Operações com ponteiros

Apenas duas operações aritméticas podem ser utilizadas com no endereço armazenado pelo ponteiro: adição e subtração

podemos apenas somar e subtrair valores INTEIROS

```
p++;
```

soma +1 no endereço armazenado no ponteiro.

```
p--;
```

subtrai 1 no endereço armazenado no ponteiro.

```
p = p+15;
```

soma +15 no endereço armazenado no ponteiro.

# Operações com ponteiros

As operações de adição e subtração no endereço dependem do tipo de dado que o ponteiro aponta.

Considere um ponteiro para inteiro, int *.

O tipo int ocupa um espaço de 4 bytes na memória.

Assim, nas operações de adição e subtração são adicionados/subtraídos 4 bytes por incremento/decremento, pois esse é o tamanho de um inteiro na memória e, portanto, é também o valor mínimo necessário para sair dessa posição reservada de memória

| Memória | | |
|---|---|---|
| posição | variável | conteúdo |
| 119 | | |
| 120 | int a | 10 |
| 121 | | |
| 122 | | |
| 123 | | |
| 124 | int b | 20 |
| 125 | | |
| 126 | | |
| 127 | | |
| 128 | char c | ‘k’ |
| 129 | char d | ‘s’ |
| 130 | | |

# Operações com ponteiros

Operações Ilegais com ponteiros

- Dividir ou multiplicar ponteiros;
- Somar o endereço de dois ponteiros;
- Não se pode adicionar ou subtrair valores dos tipos **float** ou **double** de ponteiros.

# Operações com ponteiros

Já sobre seu conteúdo apontado, valem todas as operações

(*p)++;

incrementar o conteúdo da variável apontada pelo ponteiro p;

*p = (*p) * 10;

multiplica o conteúdo da variável apontada pelo ponteiro p por 10;

```
int *p;
int c = 10;

p = &c;

(*p)++;
*p = (*p) * 10;
```

# Operações com ponteiros

Operações relacionais

**==** e **!=** para saber se dois ponteiros são iguais ou diferentes.

**>**, **<**, **>=** e **<=** para saber qual ponteiro aponta para uma posição mais alta na memória.

```
int main(){
    int *p, *p1, x, y;
    p = &x;
    p1 = &y;
    if(p == p1)
        printf("Ponteiros iguais\n");
    else
        printf("Ponteiros diferentes\n");

    return 0;
}
```

# Ponteiros Genéricos

Normalmente, um ponteiro aponta para um tipo específico de dado.

Um ponteiro genérico é um ponteiro que pode apontar para qualquer tipo de dado.

Declaração

```
void *nome_ponteiro;
```

# Ponteiros Genéricos

Exemplos

```
int main(){
    void *pp;
    int *p1, p2 = 10;
    p1 = &p2;
    //recebe o endereço de um inteiro
    pp = &p2;
    printf("Endereco em pp: %p \n",pp);
    //recebe o endereço de um ponteiro para inteiro
    pp = &p1;
    printf("Endereco em pp: %p \n",pp);
    //recebe o endereço guardado em p1 (endereço de p2)
    pp = p1;
    printf("Endereco em pp: %p \n",pp);

    return 0;
}
```

# Ponteiros Genéricos

Para acessar o **conteúdo** de um ponteiro genérico é preciso antes convertê-lo para o tipo de ponteiro com o qual se deseja trabalhar

Isso é feito vai ***type cast***

```
int main(){
    void *pp;
    int p2 = 10;
    // ponteiro genérico recebe o endereço de um
    // inteiro
    pp = &p2;
    //enta acessar o conteúdo do ponteiro genérico
    printf("Conteudo: %d\n",*pp); //ERRO
    // converte o ponteiro genérico pp para (int *)
    // antes de acessar seu conteúdo.
    printf("Conteudo: %d\n",*(int*)pp); //CORRETO

    return 0;
}
```

# Ponteiros e Arrays

Ponteiros e arrays possuem uma ligação muito forte.

Arrays são agrupamentos de dados do mesmo tipo na memória.

Quando declaramos um array, informamos ao computador para reservar uma certa quantidade de memória a fim de armazenar os elementos do array de forma sequencial.

Como resultado dessa operação, o computador nos devolve um ponteiro que aponta para o começo dessa sequência de bytes na memória.

# Ponteiros e Arrays

O nome do array (sem índice) é apenas um ponteiro que aponta para o primeiro elemento do array.

```
int vet[5] = {1,2,3,4,5};
int *p;

p = vet;
```

| Memória | | |
|---|---|---|
| posição | variável | conteúdo |
| 119 | | |
| 120 | | |
| 121 | int *p | 123 |
| 122 | | |
| 123 | int vet[5] | 1 |
| 124 | | 2 |
| 125 | | 3 |
| 126 | | 4 |
| 127 | | 5 |
| 128 | | |

# Ponteiros e Arrays

Os colchetes **[ ]** substituem o uso conjunto de operações aritméticas e de acesso ao conteúdo (operador "*") no acesso ao conteúdo de uma posição de um array ou ponteiro.

O valor entre colchetes é o deslocamento a partir da posição inicial do array.

Nesse caso, **p[2]** equivale a ***(p+2)**.

```
int main (){
    int vet[5] = {1,2,3,4,5};
    int *p;
    p = vet;

    printf("Terceiro elemento: %d ou %d",p[2],*(p+2));

    return 0;
}
```

# Ponteiros e Arrays

Nesse exemplo

```
int vet[5] = {1,2,3,4,5};
int *p;
```

Tem

```
p = vet;
```

**vet[índice]** é equivalente a ***(p+índice)**;
**vet** é equivalente a **&vet[0]**;
**&vet[índice]** é equivalente a **(vet + índice)**;

# Ponteiros e Arrays

## Usando array

```c
int main(){
    int vet[5]= {1,2,3,4,5};
    int *p = vet;
    int i;
    for (i = 0;i < 5;i++)
        printf("%d\n",p[i]);

    return 0;
}
```

## Usando ponteiro

```c
int main(){
    int vet[5]= {1,2,3,4,5};
    int *p = vet;
    int i;
    for (i = 0;i < 5;i++)
        printf("%d\n",*(p+i));

    return 0;
}
```

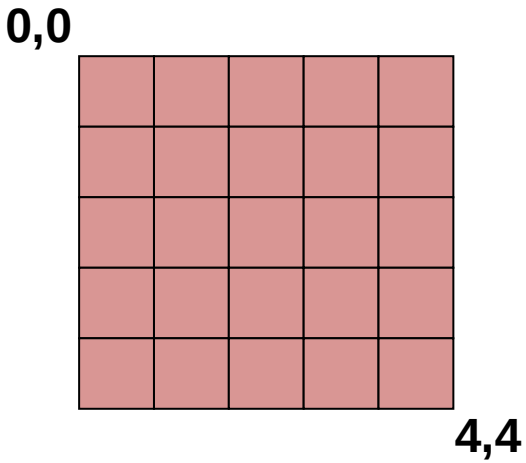
# Ponteiros e Arrays

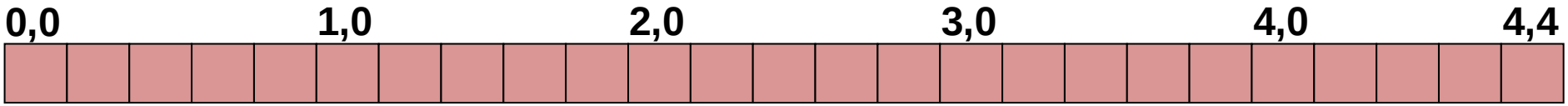
Arrays Multidimensionais

Apesar de terem mais de uma dimensão, na memória os dados são armazenados linearmente.

Ex.:

int mat[5][5];



# Ponteiros e Arrays

- Pode-se então percorrer as várias dimensões do array como se existisse apenas uma dimensão. As dimensões mais a direita mudam mais rápido

**Usando array**

```
int main(){
    int mat[2][2] = {{1,2},{3,4}};
    int i,j;
    for(i=0;i<2;i++)
        for(j=0;j<2;j++)
            printf("%d\n", mat[i][j]);

    return 0;
}
```

**Usando ponteiro**

```
int main(){
    int mat[2][2] = {{1,2},{3,4}};
    int *p = &mat[0][0];
    int i;
    for(i=0;i<4;i++)
        printf("%d\n", *(p+i));

    return 0;
}
```

# Ponteiro para struct

Existem duas abordagens para acessar o conteúdo de um ponteiro para uma struct

Abordagem 1

Devemos acessar o conteúdo do ponteiro para struct para somente depois acessar os seus campos e modificá-los.

Abordagem 2

Podemos usar o ***operador seta*** *"->"*

**`ponteiro->nome_campo`**

```
struct ponto {
    int x, y;
};

struct ponto q;
struct ponto *p;

p = &q;

(*p).x = 10;
p->y = 20;
```

# Ponteiro para ponteiro

A linguagem C permite criar ponteiros com diferentes níveis de apontamento.

É possível criar um ponteiro que aponte para outro ponteiro, criando assim vários níveis de apontamento

Assim, um ponteiro poderá apontar para outro ponteiro, que, por sua vez, aponta para outro ponteiro, que aponta para um terceiro ponteiro e assim por diante.

# Ponteiro para ponteiro

Um ponteiro para um ponteiro é como se você anotasse o endereço de um papel que tem o endereço da casa do seu amigo.

Podemos declarar um ponteiro para um ponteiro com a seguinte notação

```
tipo_ponteiro **nome_ponteiro;
```

Acesso ao conteúdo

**`**nome_ponteiro`** é o conteúdo final da variável apontada;

**`*nome_ponteiro`** é o conteúdo do ponteiro intermediário.

# Ponteiro para ponteiro

```
int x = 10;
int *p1 = &x;
int **p2 = &p1;
//Endereço em p2
printf("Endereco em p2: %p\n",p2);
//Conteudo do endereço
printf("Conteudo em *p2: %p\n",*p2);
//Conteudo do endereço do endereço
printf("Conteudo em **p2: %d\n",**p2);
```

| Memória | | |
|---|---|---|
| posição | variável | conteúdo |
| 119 | | |
| 120 | | |
| 121 | | |
| 122 | int **p2 | 124 |
| 123 | | |
| 124 | int *p1 | 126 |
| 125 | | |
| 126 | int x | 10 |
| 127 | | |

# Ponteiro para ponteiro

É a quantidade de asteriscos (*) na declaração do ponteiro que indica o número de níveis de apontamento que ele possui.

```
//variável inteira
int x;
//ponteiro para um inteiro (1 nível)
int *p1;
//ponteiro para ponteiro de inteiro (2 níveis)
int **p2;
//ponteiro para ponteiro para ponteiro de inteiro (3 níveis)
int ***p3;
```

# Ponteiro para ponteiro

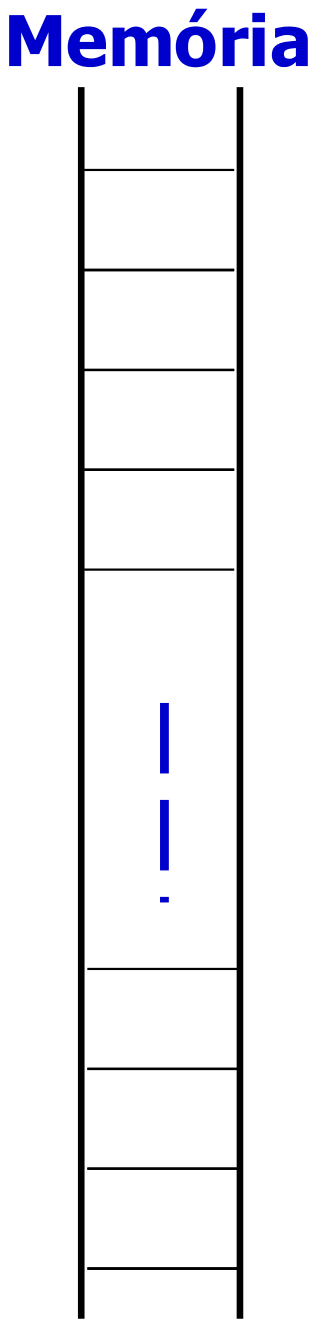
Conceito de “ponteiro para ponteiro”:

```
char letra = 'a';
char *p1;
char **p2;
char ***p3;

p1 = &letra;
p2 = &p1;
p3 = &p2;
```

| Memória | | |
|---|---|---|
| posição | variável | conteúdo |
| 119 | | |
| 120 | char ***p3 | 122 |
| 121 | | |
| 122 | char **p2 | 124 |
| 123 | | |
| 124 | char *p1 | 126 |
| 125 | | |
| 126 | char letra | ‘a’ |
| 127 | | |

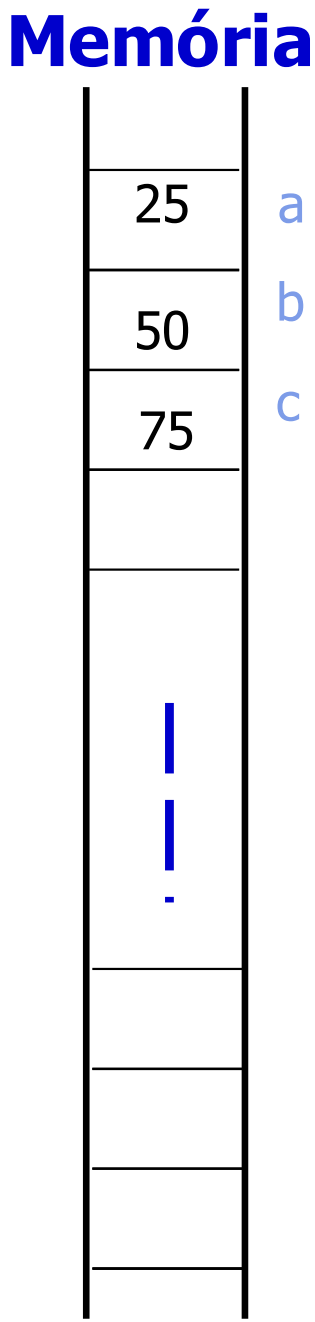
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



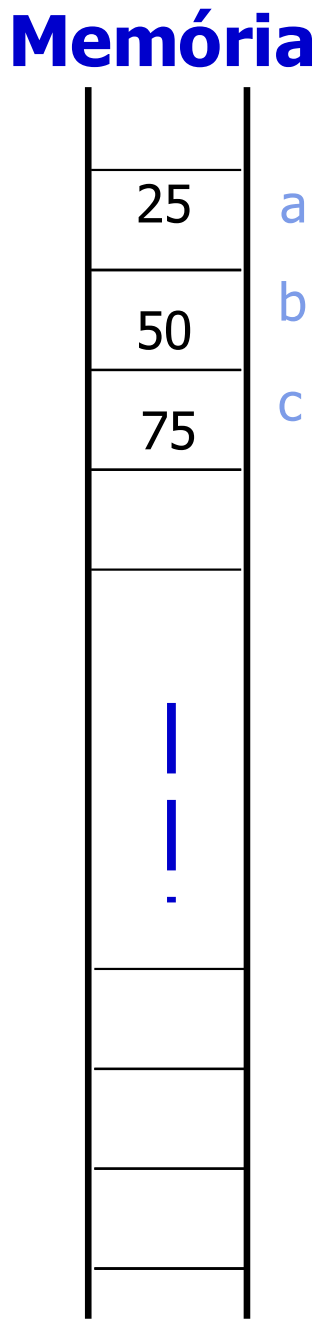
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



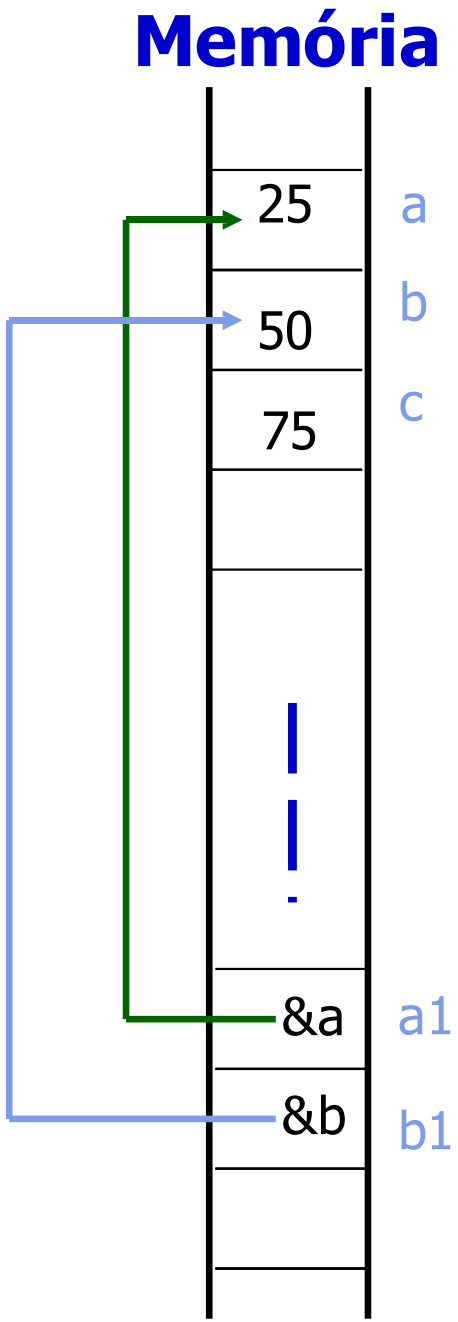
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



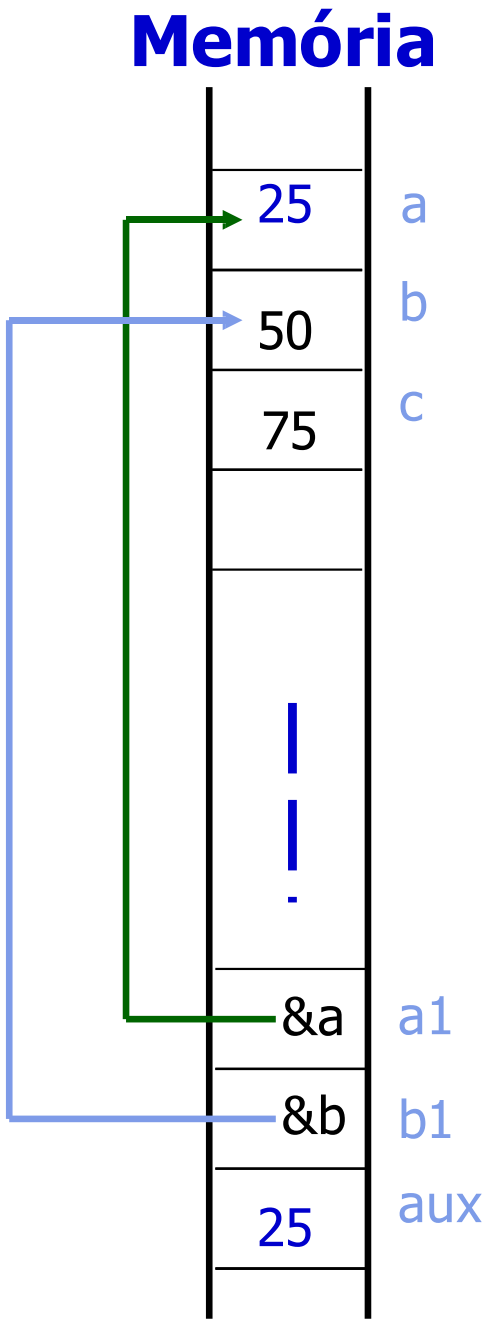
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



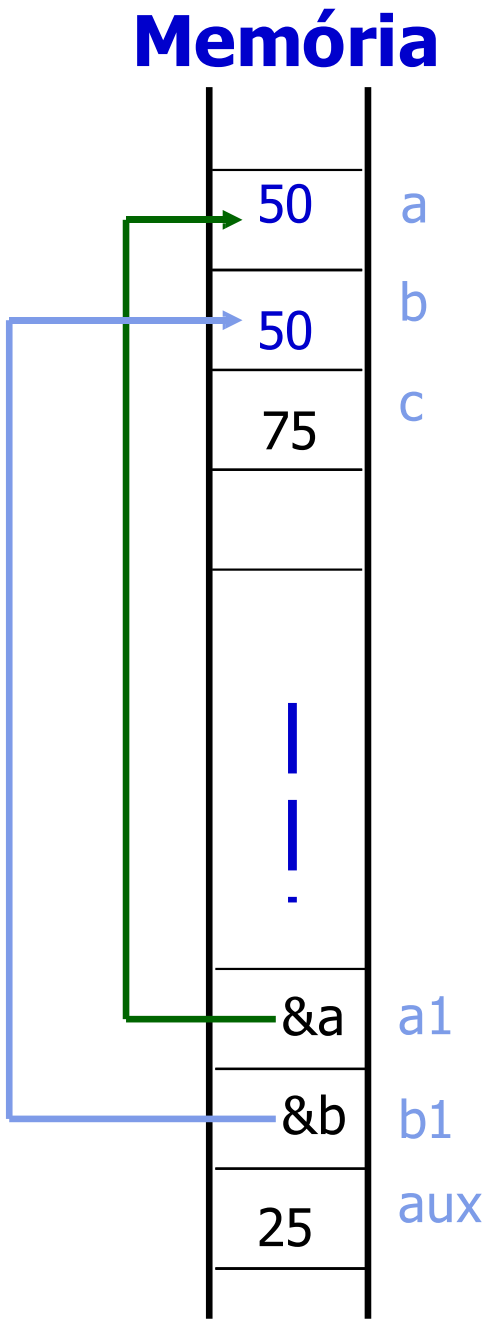
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



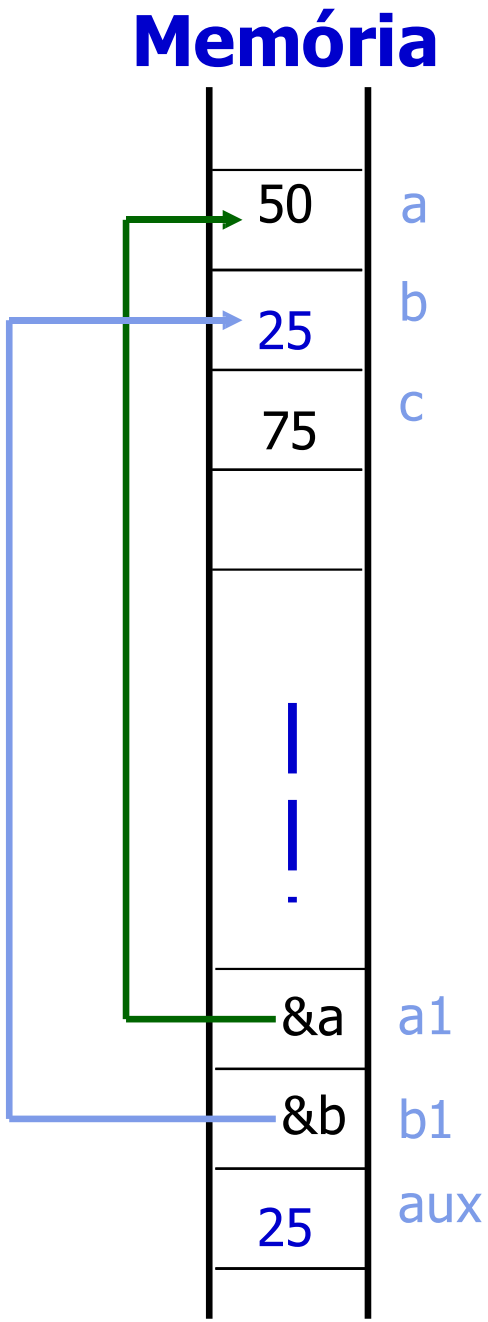
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



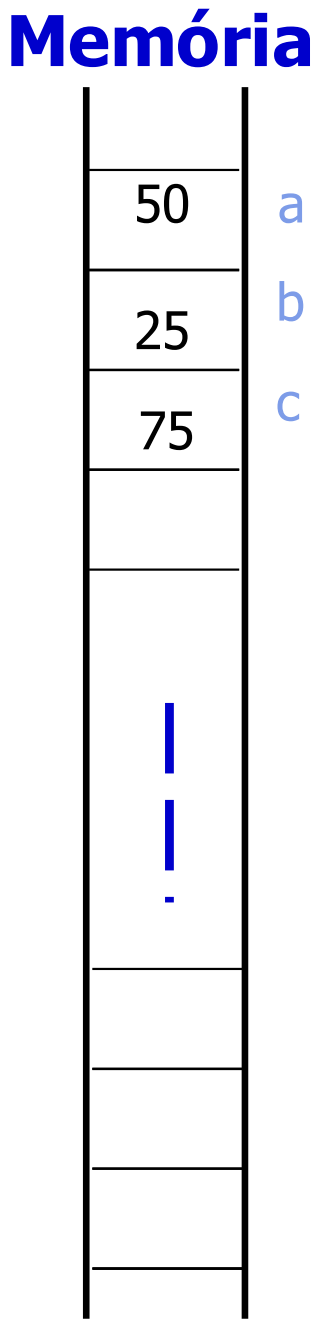
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



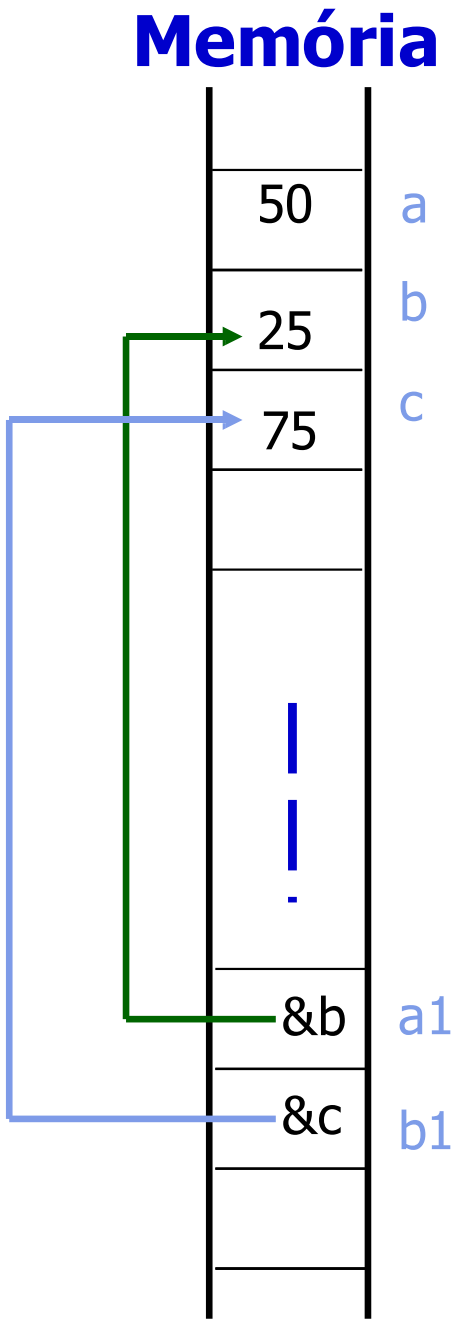
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



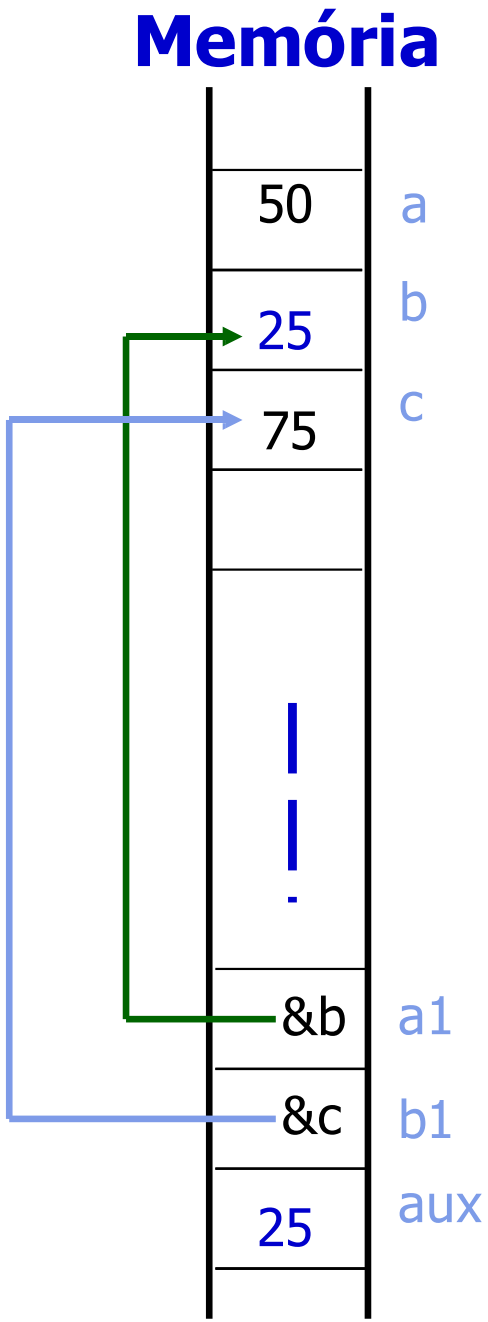
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



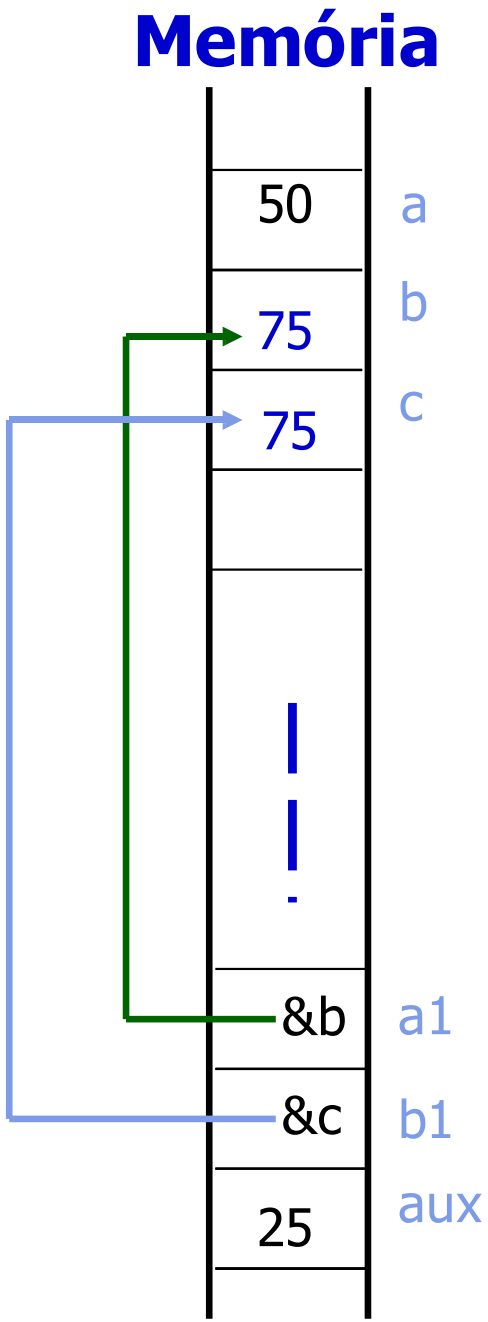
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



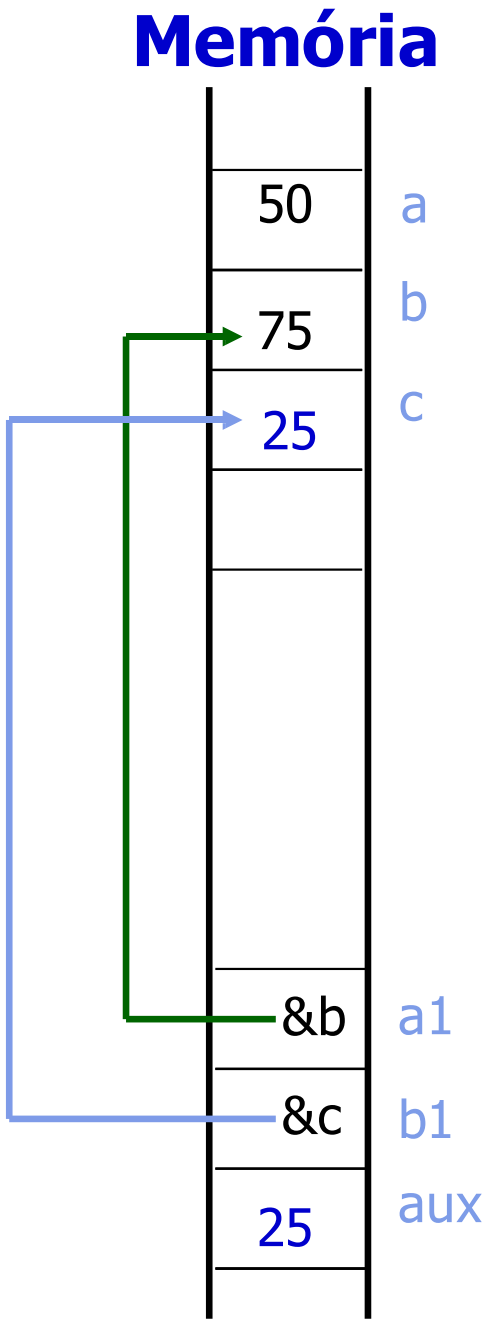
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



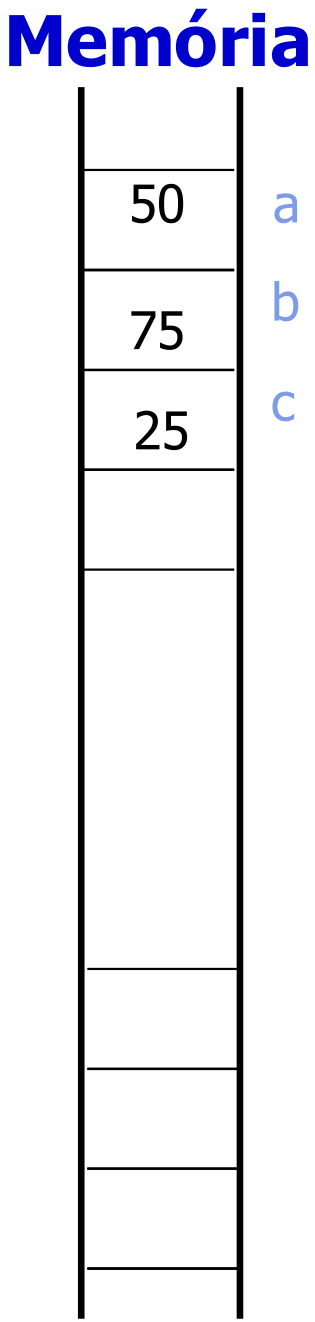
# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



# Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



# Exercícios

1) O que será impresso no programa abaixo?

```
#include <stdio.h>

void calculo (int *p, int *q)
{
 *p = *p * 10;
 *q = *q + 10;
}

int main()
{
   int x = 2, y = 5;
   calculo(&x,&y);
   printf("%d - %d",x,y);
   return 0;
}
```

# Exercícios

2) Qual a saída do programa abaixo:

```
#include <stdio.h>
int calculo (int p, int q)
{
 p = p * 10;
 q = q + 10;
 return(p + q);
}

int main()
{
   int x = 2, y = 5;
   printf("%d %d %d",x,y,calculo(x,y));
   return 0;
}
```

# Exercícios

3) Qual a saída do programa abaixo:

```
#include <stdio.h>

int calculo (int p, int *q)
{
  p = p * 10;
  *q = *q + 10;
  return(p);
}

int main()
{
   int x = 2, y = 5, z;
   z = calculo(x,&y);
   printf("%d  %d  %d",x,y,z);
   return 0;
}
```

# Exercícios

4) Qual a saída do programa abaixo:

```
#include <stdio.h>

int cal(int p, int *q, int *r)
{
  p = p * 10;
  *q = *q + 10;
  *r = *r - 10;
  return(p);
}

int main()
{
  int x = 2, y = 5, z = 3, r;
  r = cal(x,&y,&z);
  printf("%d  %d %d %d",x,y,z,r);
  return 0;
}
```

# *Vetores*

- O vetor é uma estrutura:
  - Homogênea
  - Estática

- Todas as componentes são de um mesmo tipo e seu tamanho permanece o mesmo durante toda a execução do programa.

# *Vetores: Declaração*

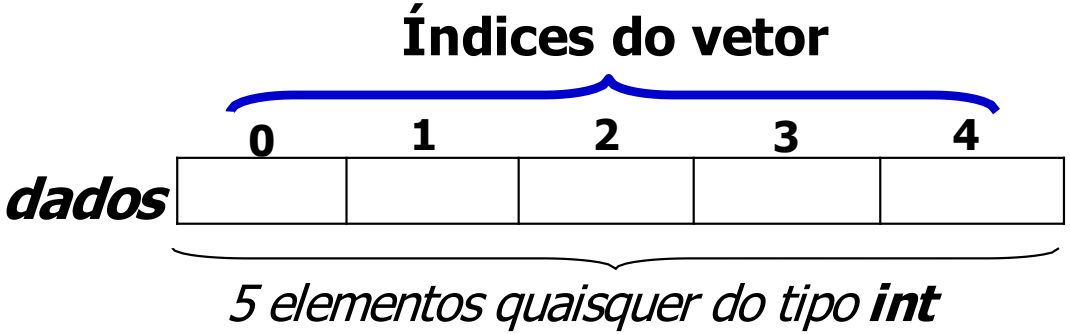
- A sintaxe em C para declaração de variável do tipo vetor é a seguinte

  tipo_primitivo identificador[qtde_elementos];

Exemplo:

// vetor com 5 (0 a 4) elementos do tipo int

int dados[5];



# *Vetores: Referência (Manipulação)*

- Cada um dos elementos de um vetor é **referenciado individualmente por meio de um número inteiro entre colchetes após o nome do vetor**.

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| ***valores*** | **3.6** | **2.7** | **4.2** | **7.9** | **1.2** |

Exemplos:

- **X = valores[1]; //atribui a x o valor da posição 1 do vetor valores**
- **Y = valores[4]; //atribui a x o valor da posição 4 do vetor valores**
- **valores[0] = 3.2; // a posição zero do vetor valores recebe o valor 3.2**

# *Vetores: Inicialização*

É possível fornecer valores a cada elemento de um vetor no momento da sua declaração.

Exemplo:

**float valores[5] = {3.6, 2.7, 4.2, 7.9, 1.2};**

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| ***valores*** | 3.6 | 2.7 | 4.2 | 7.9 | 1.2 |

**float nros[5] = {0.0}; *// como fica o vetor????***

# *Vetores: Inicialização*

Observações:

- Os vetores NÃO são automaticamente inicializados!!!

- Se houver menos valores do que o número de elementos do vetor , os elementos restantes são inicializados automaticamente com o valor zero.

```
int n[5] = {32, 64, 27};
```

- A seguinte declaração causa um erro de sintaxe:

```
int n[5] = {32, 64, 27, 18, 95, 14};
```

- O tamanho do vetores pode ser omitido:

```
int n[ ] = {1, 2, 3, 4, 5};
```

# Vetores: Exemplo

- Leia um vetor de 10 posições (inteiros) e imprima-o na ordem invertida (da última para a primeira posição).

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int numeros[10],i;
   for (i=0; i<10;i++){
       printf("Digite valor %d: ",i);
       scanf("%d", &numeros[i]);
   }
   printf("Vetor na ordem invertida:\n");
   for (i=9; i>=0; i--)
      printf("%d\n",numeros[i]);
   return 0;

}
```

# Vetores e Subrotinas

- Em C, vetores são passados sempre por referência.

- Ou seja, as modificações feitas na subrotina refletem nos dados do vetor passado como parâmetro pela função chamadora.

# *Exercícios*

1) Desenvolva um programa que leia um vetor de números reais, um escalar e imprima o resultado da multiplicação do vetor pelo escalar.

2) Faça um procedimento que faça a leitura um vetor de 10 elementos inteiros e imprima somente os valores armazenados nos índices pares.

3) Faça um programa que leia um vetor com 15 valores reais. A seguir, encontre o menor elemento do vetor e a sua posição dentro do vetor, mostrando: “O menor elemento do vetor esta na posição XXXX e tem o valor YYYYY.”

4) Faça um programa que leia um vetor de 15 posições (reais) e depois um valor a ser procurado no vetor. Imprima se o valor foi ou não encontrado e a quantidade de vezes que o valor está presente no vetor.

# *Exercícios*

5) Faça uma função que receba um vetor de números inteiros e um valor inteiro. A função deverá procurar este segundo valor neste vetor e retornar seu índice se for encontrado. Se o elemento não for encontrado, retornar -1.

6) Dada uma tabela com as notas de uma turma de 20 alunos, faça funções que retornem:

a) A média da turma.

b) a quantidade de alunos aprovados (>=60)

c) a quantidade de alunso reprovados.(< 60)

7) Faça um programa que leia um conjunto de 20 valores e armazene-os num vetor V. Particione-o em dois outros vetores, P e I, conforme os valores de V forem pares ou ímpares. No final, imprima os valores dos 3 vetores.

# *Exercícios*

8) Faça um programa que leia um vetor G[13] que é o gabarito de um teste, contendo os valores 0,1 ou 2.
Ler a seguir, 5 cartões de respostas (R[13]) e depois da leitura imprimir quantos acertos cada cartão teve.

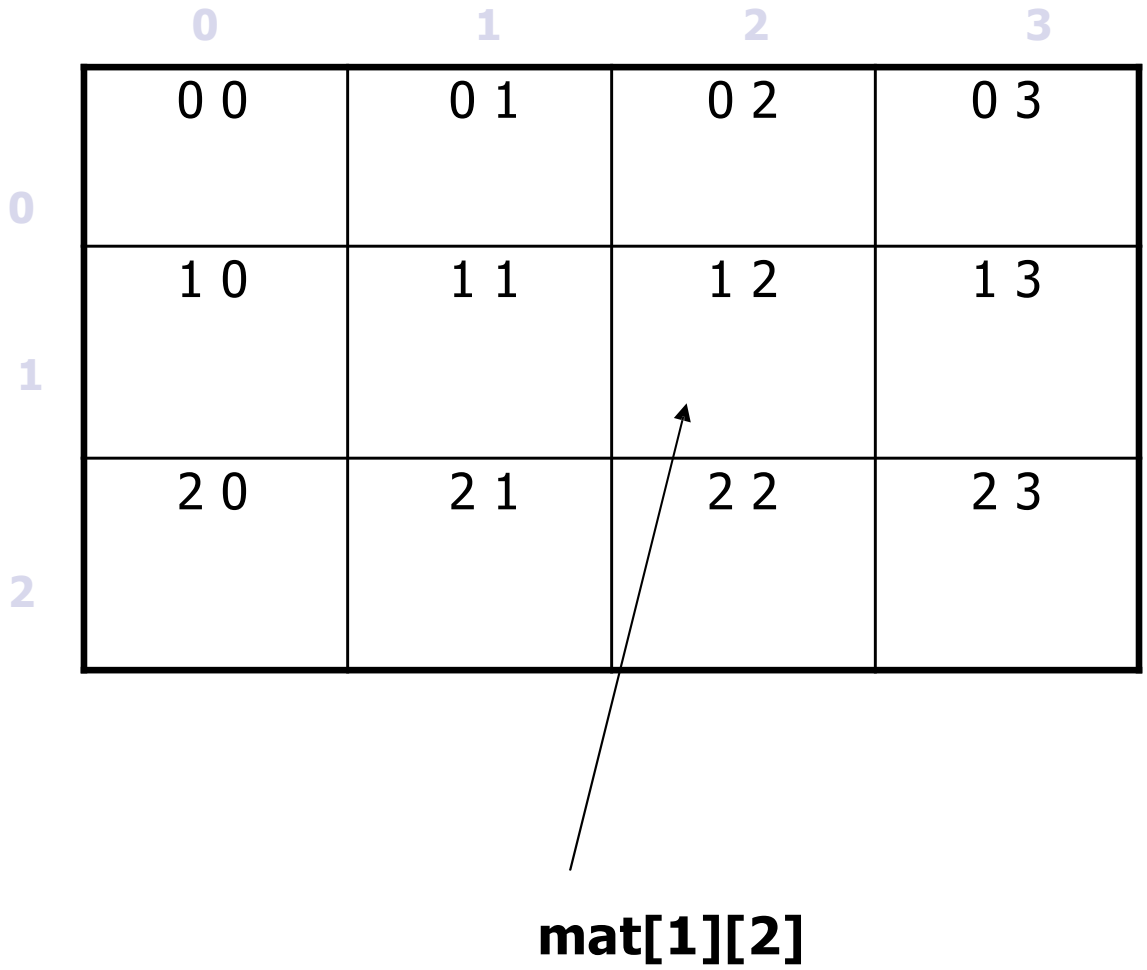
Faça o teste através de funções.

9) Com relação ao exercício anterior, calcule e mostre o percentual dos apostadores que fizeram de 10 a 13 pontos e o percentual dos apostadores que fizeram menos do que 10 pontos.

10) Faça um programa que leia um vetor de valores inteiros e imprima-o na ordem crescente. O vetor deve ter tamanho N (utilize a diretiva #define).

# *Declaração de matrizes*

int mat[3][4];



# *Declaração de matrizes*

- **Declaração de matrizes com mais de 2 dimensões**

float mat2[3][5][2];

int mat3[3][5][2][7];

etc…

- **Operações em matrizes**

mat2[2][4][1] = 8.6;

- mat3[0][0][0][1] = 5;

# *Matrizes: inicialização*

Pode-se fornecer valores de cada elemento de uma matriz na declaração, da mesma forma que nos vetores

Exemplo:

```
float num[2][3] = {{3.6, 2.7,1.5},{5.0,4.1,2.3}};
```

Ou seja, fornece-se os valores linha a linha.

```
int val[5][2] = {{3,7}} // inicializou a primeira com os
        //valores 3 e 7. As demais receberão zero.
```

# *Matrizes: inicialização*

Observações:

- As matrizes não são inicializadas automaticamente

- Se houver menos valores do que o número de elementos da matriz, os elementos restantes são inicializados automaticamente com o valor zero.

```
int num[5][3] = {{32, 64, 27}};
```

```
int n[4][4] = {{0}}; //todos elementos são nulos
```

- A seguinte declaração causa um erro de sintaxe:

```
int n[2][5] = {{32, 64, 27, 18, 95, 14},{12,15,43,17,67,31}};
```

# *Matrizes: exemplos*

O programa abaixo inicializa os elementos de uma matriz **m** com os valores iguais a soma dos índices de cada elemento e imprime cada valor.

```
#include <stdio.h>

int main()
{
  int m[3][2], i,j;

  for (i=0; i < 3; i++)
    for(j=0; j < 2; j++)
    {
      m[i][j] = i+j;
      printf("i=%d j=%d elemento=%d\n",i,j,m[i][j]);
    }
  return 0;
}
```

```
i= 0 j= 0 elemento=0
i= 0 j= 1 elemento=1
i= 1 j= 0 elemento=1
i= 1 j= 1 elemento=2
i= 2 j= 0 elemento=2
i= 2 j= 1 elemento=3
```

# *Matrizes e subrotinas*

- Matrizes serão passadas para subrotinas da mesma forma como os vetores são passados, a menos de um detalhe: **apenas a primeira dimensão pode ser omitida.**

  **Ex.:**

```
void imprimeMatriz(float m[][3])
```

**ou**

```
void imprimeMatriz(float m[3][3])
```

- **Havendo mais dimensões, a mesma regra deverá ser seguida.**

# Exercícios

1)Faça um procedimento para calcular a multiplicação de uma matriz 3 x 4 por um escalar. Faça também um procedimento capaz de imprimir esta matriz. Ao final, desenvolva uma função principal onde será criada e lida uma matriz 3 x 4. Faça nesta função a chamada dos dois procedimentos criados anteriormente.

2)Faça um programa que leia uma matriz 7 x 5 e imprima a terceira coluna.

3)Faça um programa que leia uma matriz quadrada de dimensão 10, uma função que encontre o maior valor desta matriz e uma função que encontre o menor valor. Imprima os valores encontrados na função principal.

4)Faça um programa que leia uma matriz 6 x 3 e um procedimento que gere duas matrizes 3 x 3, a primeira com as 3 primeiras linhas e a outra com as restantes.

# Exercícios

5)Faça um programa que leia uma matriz de caracteres 5 x 10 e uma função que conte quantas letras “a” aparecem na matriz. Você poderá fazer a leitura letra a letra ou considerar que cada linha da matriz é uma *string*.

6)Faça um programa que leia 3 vetores reais de dimensão 10 e um procedimento que gere uma matriz (10 x 3) onde cada coluna é dada por um destes vetores.

7)Crie uma matriz tridimensional onde as linhas indicam as notas de matemática, história e geografia em três provas de 10 alunos e crie uma função que verifique quantos alunos passaram, ou seja, os que tenham média aritmética > 60 nas 3 disciplinas.

# Exercícios

8)Faça um programa para ler a quantidade de um total de 5 produtos que uma empresa tem em suas 7 lojas e imprimir em uma tabela:

a) o total de cada produto nestas lojas

b) a loja que tem menos produtos

9) Idem ao exercício 1, agora com o seguinte “Menu”:

*Escolha uma opção de cálculo para matrizes:*
*1)Soma*
*2) Diferença*
*3) Transposta*
*4) Produto*
*5)Sair*
*Opção:___*

Na opção (3) o usuário só precisa fornecer uma matriz, nas opções (1), (2) e (4) o usuário deve fornecer duas matrizes.

Observação, a matriz deve ser 3x3.

# Exercícios

10) Implemente as subrotinas abaixo para que o programa funcione corretamente.

```
#include <stdio.h>
#define MAX 10
int  main()
{
   float a[MAX][MAX], b[MAX][MAX];
   int m, n;
   printf("\nDigite o no de linhas e de colunas das matrizes: ");
   scanf("%d,%d",&m,&n);
   printf("\nForneça os elementos da matriz a:\n");
   lematriz(a,m,n);    // leitura da matriz a
   printf("\nForneça os elementos da matriz b:\n");
   lematriz(b,m,n);   // leitura da matriz b
   copiazera(a,b,m,n); // a matriz a "recebe" b e b é zerada.
   printf("\nMatriz a:\n");
   imprimematriz(a,m,n);
   printf("\nMatriz b:\n");
   imprimematriz(b,m,n);
   return 0;
}
```

# Estruturas heterogêneas

- Até agora vimos as estruturas de dados homogêneas: **vetores**, **matrizes** e ***strings***.

- Nestas estruturas todos os elementos da estrutura são de tipos de dados primitivos: **inteiro, real, caractere**.

# Estruturas heterogêneas

- No entanto, em muitos casos, necessitamos armazenar conjuntos de informações relacionadas, formados por diversos tipos de dados primitivos.

- Exemplos:
  - Endereço;
  - Fichas com dados pessoais de um cliente;
  - Fichas com dados de um produto.

# Variáveis compostas

- Quando uma determinada estrutura de dados for composta por **diversos tipos diferentes,** primitivos ou não, temos um conjunto heterogêneo de dados.

- Essas variáveis são chamadas de **variáveis compostas heterogêneas**.

- Estas variáveis compostas são chamadas de **estruturas** (ou ***structs*** na linguagem C).

# Definição de estrutura

- Uma estrutura pode ser definida como uma coleção de uma ou mais variáveis relacionadas (campos), **onde cada variável pode ser de um tipo distinto**.

- Exemplo:

  empregado

  (4 campos)

# Estruturas - Definição

- Sintaxe para definir uma estrutura com **n** campos em C:

```
struct NomeDaEstrutura
{
  tipo1 identificador1;
  tipo2 identificador2;
    ...
  tipoN identificadorN;
};
```

# Estruturas - Declaração

- No entanto, dessa forma, é necessário usar a palavra-chave **struct** toda vez que uma variável é declarada:

```
struct <nomeEstrutura> var1, ..., varN;
```

- Exemplo:

```
// Definicao da estrutura
struct Aluno
{
  char nome[100];
  int matricula;
  int idade;
  char cidade[50];
  char sexo;
};
```

```
// Declaracao de variaveis
struct Aluno fulano;
struct Aluno a1, a2, a3;
```

# Estruturas - Declaração

- Note que para variáveis do tipo estrutura a palavra chave **struct** deve vir antes do nome da estrutura.

```
// Tipos primitivos
int valor;
float x, y, z;
char letra;
```

```
// Variaveis estruturas
struct Aluno fulano;
struct Aluno a1, a2, a3;
```

- Existe uma outra possibilidade, mais simples, para declarar variáveis de estruturas.

# Comando `typedef`

- O comando **`typedef`**, pode ser usado para dar um novo nome para algum tipo de dados.
- Uso: `typedef <nomeAntigo> <nomeNovo>;`
- Exemplos:

```
typedef int Inteiro;
typedef float Real;
```

- Declarando variáveis

```
Inteiro n;
Real x;
```

# Estruturas - Declaração

- Assim, com o comando **typedef**, pode-se definir estruturas e declarar variáveis da seguinte forma:

```
typedef struct est_Aluno   Aluno;
```

Nome antigo

Nome novo

- Exemplo

```
// Definicao da estrutura
struct est_Aluno
{
  char nome[100];
  int matricula;
  int idade;
  char cidade[50];
  char sexo;
};
```

```
typedef struct est_Aluno Aluno;

// Declaracao de variaveis
Aluno a1, a2, a3;
```

# Estruturas - Declaração

- A seguinte forma também pode ser utilizada:

```
// Definicao da estrutura
typedef struct
{
   char nome[100];
   int matricula;
   int idade;
   char cidade[50];
   char sexo;
} Aluno;

// Declaracao de variaveis
Aluno a1, a2, a3;
```

Esta será a forma adotada na disciplina.

# Definição e Declaração

- A definição de um tipo estrutura deve ficar **fora** do programa principal (**`main`**) e de qualquer função.

- A declaração de uma variável do tipo estrutura deve ficar **dentro** de alguma função (seja ela a main ou outra qualquer).

```
#include <stdio.h>

typedef struct
{
  char nome[100];
  int matricula;
  int idade;
  char cidade[50];
  char sexo;
} Aluno;



int main()
{
  Aluno a1, a2, a3;
  // ...
  // processamento
  // ...
  return 0;
}
```

# Estruturas: Manipulação

- Cada campo de uma estrutura pode armazenar um valor (como uma variável). Seu valor pode ser acessado, modificado, impresso, etc.
- Campos são acessados usando o operador de acesso **ponto** (.) entre o **nome da variável** declarada como estrutura e o **nome do campo**.

```
typedef struct
{
  char nome[100];
  int matricula;
  int idade;
  char cidade[50];
  char sexo;
} Aluno;
```

# Estruturas: Manipulação

- No programa são criadas duas variáveis **`Aluno`**.
- Cada variável **`Aluno`** (**`a1`** e **`a2`**) tem nome, matricula, idade, cidade e sexo.
- O uso de **`a1.idade`** permite acessar ou modificar o campo **`idade`** da variável **`a1`**.

```
typedef struct
{
  char nome[100];
  int matricula;
  int idade;
  char cidade[50];
  char sexo;
} Aluno;
```

```
int main()
{
  Aluno a1, a2;

  gets(a1.nome);
  a1.idade = 20;
  printf("Matricula %d", a2.matricula);
  // ...
  return 0;
}
```

# Estruturas: Manipulação

- Neste exemplo, serão lidos matrícula e nome de um funcionário.

```
#include <stdio.h>
typedef struct {
  char matricula[15];
  char nome[100];
  int setor;
} Funcionario;

int main()
{
  Funcionario f1;
  gets(f1.matricula);
  gets(f1.nome);
  scanf("%d", &f1.setor);
  puts("Informacoes lidas:");
  puts(f1.matricula);
  puts(f1.nome);
  printf("%d\n", f1.setor);
  return 0;
}
```

# Exercícios

1) Defina uma estrutura para representar as informações de um cartão de crédito. Você precisa apenas criar a estrutura com os dados e tipos apropriados. Não é necessário criar um programa para utilizá-la.

2) Defina uma estrutura chamada `medidas` para representar o peso a altura de uma pessoa.

3) Considerando a estrutura do exercício (2) e a declaração:

```
medidas joao, maria;
```

→ Escreva instruções para atribuir: 1.78 à altura de `joao`; 1.64 à altura de `maria`; 75 ao peso de `joao`; 59 ao peso de `maria`.

→ Escreva um conjunto de instruções para imprimir a média das alturas e a média dos pesos de `joao` e `maria`.

# Estruturas aninhadas

- Definição de uma estrutura onde um de seus campos é outra estrutura:

```
typedef struct
{
  char nome[50];
  int idade;
  endereco end;
} ficha_pessoal;
```

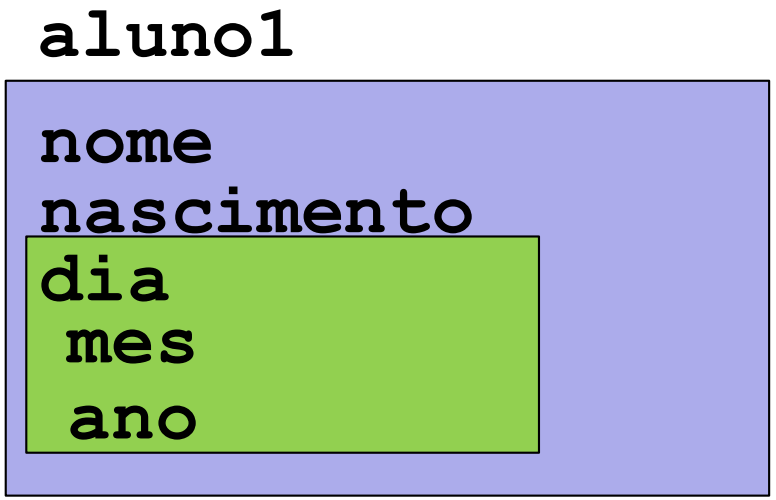
Repare que um campo do tipo **endereco** é declarado dentro de **ficha_pessoal**.

- Declaração de variáveis do tipo definido acima:

```
ficha_pessoal ficha1, ficha2;
```

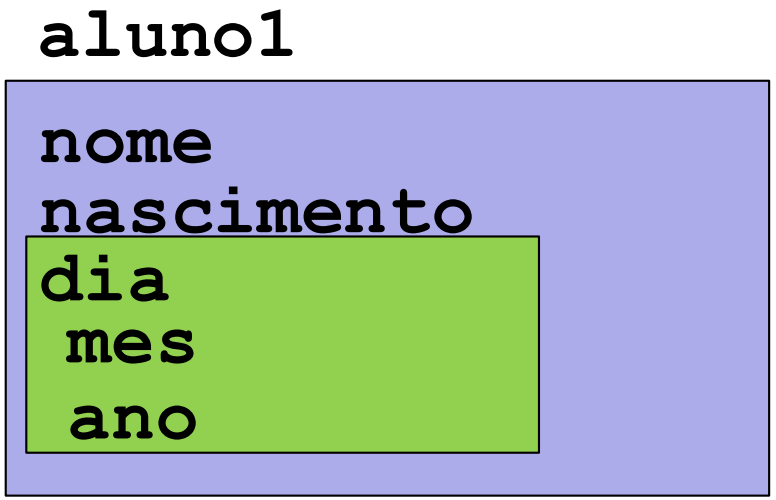
# Estruturas aninhadas

```
typedef struct {
  int dia,mes,ano;
} Data;

typedef struct {
  char nome[150];
  Data nascimento;
} Aluno;

int main() {
  Aluno aluno1;
  …
```



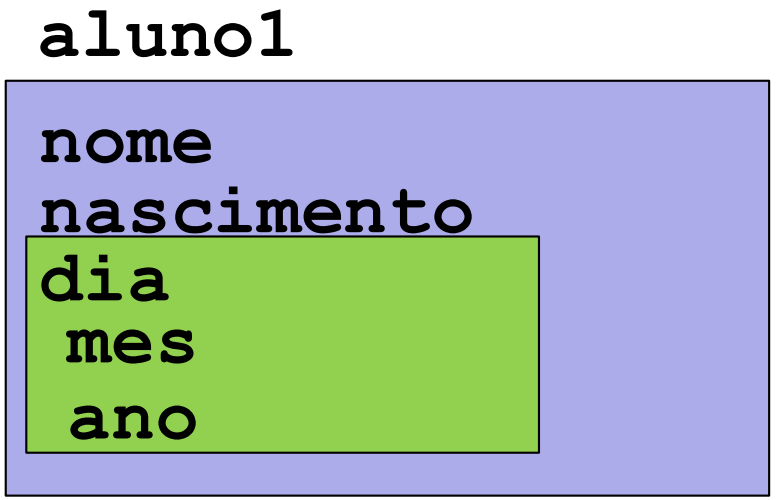
# Estruturas aninhadas

```
typedef struct {
  int dia,mes,ano;
} Data;

typedef struct {
  char nome[150];
  Data nascimento;
} Aluno;

int main() {
  Aluno aluno1;
  gets(aluno1.nome);
  …
```



# Estruturas aninhadas

```
typedef struct {
   int dia,mes,ano;
} Data;

typedef struct {
   char nome[150];
   Data nascimento;
} Aluno;

int main() {
   Aluno aluno1;
   gets(aluno1.nome);
   aluno1.nascimento.dia = 2;
   …
```



# Estruturas aninhadas

```
typedef struct {
  int dia,mes,ano;
} Data;

typedef struct {
  char nome[150],cpf[12];
  Data nascimento;
} Responsavel;

typedef struct {
  char nome[150];
  Data nascimento;
  Responsavel mae,pai;
} Aluno;

int main() {
  Aluno aluno1;
  …
```

# Estruturas aninhadas

```
typedef struct {
  int dia,mes,ano;
} Data;

typedef struct {
  char nome[150],cpf[12];
  Data nascimento;
} Responsavel;

typedef struct {
  char nome[150];
  Data nascimento;
  Responsavel mae,pai;
} Aluno;

int main() {
  Aluno aluno1;
  gets(aluno1.nome);
  aluno1.nascimento.dia = 2;
  …
```

# Estruturas aninhadas

```
typedef struct {
  int dia,mes,ano;
} Data;

typedef struct {
  char nome[150],cpf[12];
  Data nascimento;
} Responsavel;

typedef struct {
  char nome[150];
  Data nascimento;
  Responsavel mae,pai;
} Aluno;

int main() {
  Aluno aluno1;
  gets(aluno1.nome);
  aluno1.nascimento.dia = 2;
  aluno1.mae.nascimento.ano = 1975;
  printf("%s", aluno1.pai.cpf); …
```

# Estruturas com vetor

- Em alguns casos o tipo estrutura possui vetores como um dos seus campos.

```
typedef struct
{
   int vetorA[4];
   char vetorB[10];
} Dados;
```

- O acesso a estes campos é feito da mesma maneira como acesso direto a um vetor.

```
Dados registro;
registro.vetorA[2] = 100;
gets(registro.vetorB);
```

# Exercício

4) Faça um programa (função principal) para leitura, via teclado, dos dados de um atleta.
Os dados a serem guardados na estrutura atleta são os seguintes: nome, medidas (estrutura definida no exercício 2), esporte praticado e idade.
Ao final, imprima estas informações na tela.

# Atribuição de estruturas

- Uma das vantagens ao utilizarmos estruturas é a possibilidade de copiarmos toda a informação de uma estrutura para outra do mesmo tipo com uma atribuição simples:

```
typedef struct {
  int X;
  int Y;
} Coordenadas;

int main ()
{
   Coordenadas primeira, segunda;
   primeira.X = 20;
   primeira.Y = 30;
   segunda = primeira;
   printf("%d %d", segunda.X, segunda.Y);
   return 0;
}
```

# Exemplo completo

- Considere as informações de um aluno que tem NOME e 4 notas como campos de uma estrutura; veja *layout* abaixo.

Cadastro de notas escolares

Nome:_____

| Notas | | | |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| | | | |

- Desenvolver um algoritmo para ler e imprimir o nome e as notas de um aluno.

# Exemplo completo

- Ideia básica do algoritmo :
  1. Definir estrutura
  2. Declarar variáveis
  3. Ler os dados de um aluno
  4. Imprimir os dados do aluno

# Exemplo completo

1. Definir estrutura

```
typedef struct
{
   char nome[40];
   float notas[4];
} Aluno;
```

# Exemplo completo

1. Definir estrutura
2. Declarar variáveis

```
typedef struct
{
   char nome[40];
   float notas[4];
} Aluno;
```

```
int main()
{
    Aluno a; // dados do aluno
    int i;// contador  para percorrer o vetor de notas
    …
```

# Exemplo completo

1. Definir estrutura
2. Declarar variáveis
3. Ler dados de um aluno

```
typedef struct
{
   char nome[40];
   float notas[4];
} Aluno;

int main()
{
    Aluno a;
    int i;
    …
```

```
gets(a.nome);
for (i = 0 ; i <= 3; i++ )
{
   scanf("%f", &a.notas[i]);
}
```

# Exemplo completo

1. Definir estrutura
2. Declarar variáveis
3. Ler dados de um aluno
4. Imprimir os dados

```c
typedef struct
{
  char nome[40];
  float notas[4];
} Aluno;

int main()
{
   Aluno a;
   int i;
   gets(a.nome);
   for (i = 0 ; i <= 3; i++ )
   {
     scanf("%f", &a.notas[i]);
   }
```

```c
puts(a.nome);
for (i = 0 ; i <= 3; i++ )
{
  printf("%f", a.notas[i]);
}
```

# Exemplo completo

```
typedef struct {
  char nome[40];
  float notas[4];
} Aluno;

int main()
{
  Aluno a;
  int i;
  gets(a.nome);
  for (i = 0 ; i <= 3; i++ )
  {
    scanf("%f", &a.notas[i]);
  }

  puts(a.nome);
  for (i = 0 ; i <= 3; i++ )
  {
    printf("%f", a.notas[i]);
  }
  return 0;
}
```

# Estruturas e funções

- Como qualquer outra variável, uma variável do tipo estrutura pode ser usada como **parâmetro** ou pode ser o **valor de retorno** de uma função.

```
typedef struct
{
    int  valorA;
    int  valorB;
} MinhaEstrutura;
```

```
void imprime(MinhaEstrutura e1)
{
    printf("\n%d",e1.valorA);
    printf("\n%d",e1.valorB);
    e1.valorB = e1.valorA;
}
MinhaEstrutura  novaEstrutura( )
{
    MinhaEstrutura novaVar;
    scanf("%d",&novaVar.valorA);
    scanf("%d",&novaVar.valorB);
    return novaVar;
}
int main()
{
    MinhaEstrutura var;
    var = novaEstrutura();
    imprime(var);
    return 0;
}
```

Tipo de retorno da função

# Estruturas e funções

- Como qualquer outra variável, uma variável do tipo estrutura pode ser usada como **parâmetro** ou pode ser o **valor de retorno** de uma função.

```
typedef struct
{
    int valorA;
    int valorB;
} MinhaEstrutura;
```

```
void imprime(MinhaEstrutura e1)
{
    printf("\n%d",e1.valorA);
    printf("\n%d",e1.valorB);
    e1.valorB = e1.valorA;
}
MinhaEstrutura novaEstrutura( )
{
    MinhaEstrutura novaVar;
    scanf("%d",&novaVar.valorA);
    scanf("%d",&novaVar.valorB);
    return novaVar;
}
int main()
{
    MinhaEstrutura var;
    var = novaEstrutura();
    imprime(var);
    return 0;
}
```

O valor de **var** é copiado para **e1**

A variável **e1** é alterada. Observe que **e1** é uma cópia de **var**. Assim, **var** não é alterada.

A variável **var** não é alterada na função **imprime**

# Estruturas e funções

- Quando o ponteiro para uma estrutura é passado como parâmetro a forma de acesso muda

```
typedef struct
{
    int  valorA;
    int  valorB;
} MinhaEstrutura;
```

```
void imprime(MinhaEstrutura *e1)
{
    printf("\n%d",e1->valorA);
    printf("\n%d",e1->valorB);
    e1->valorB = e1->valorA;
}
MinhaEstrutura novaEstrutura( )
{
    MinhaEstrutura novaVar;
    scanf("%d",&novaVar.valorA);
    scanf("%d",&novaVar.valorB);
    return novaVar;
}
int main()
{
    MinhaEstrutura var;
    var = novaEstrutura();
    imprime(&var);
    return 0;
}
```

# Vetores de estruturas

- Pode-se criar vetores de estruturas como se criam vetores de tipos primitivos.

- Os programas apresentados até o momento só fizeram menção a uma única instância da estrutura.

- É necessário possuir uma definição da estrutura antes de declarar um vetor de estrutura.

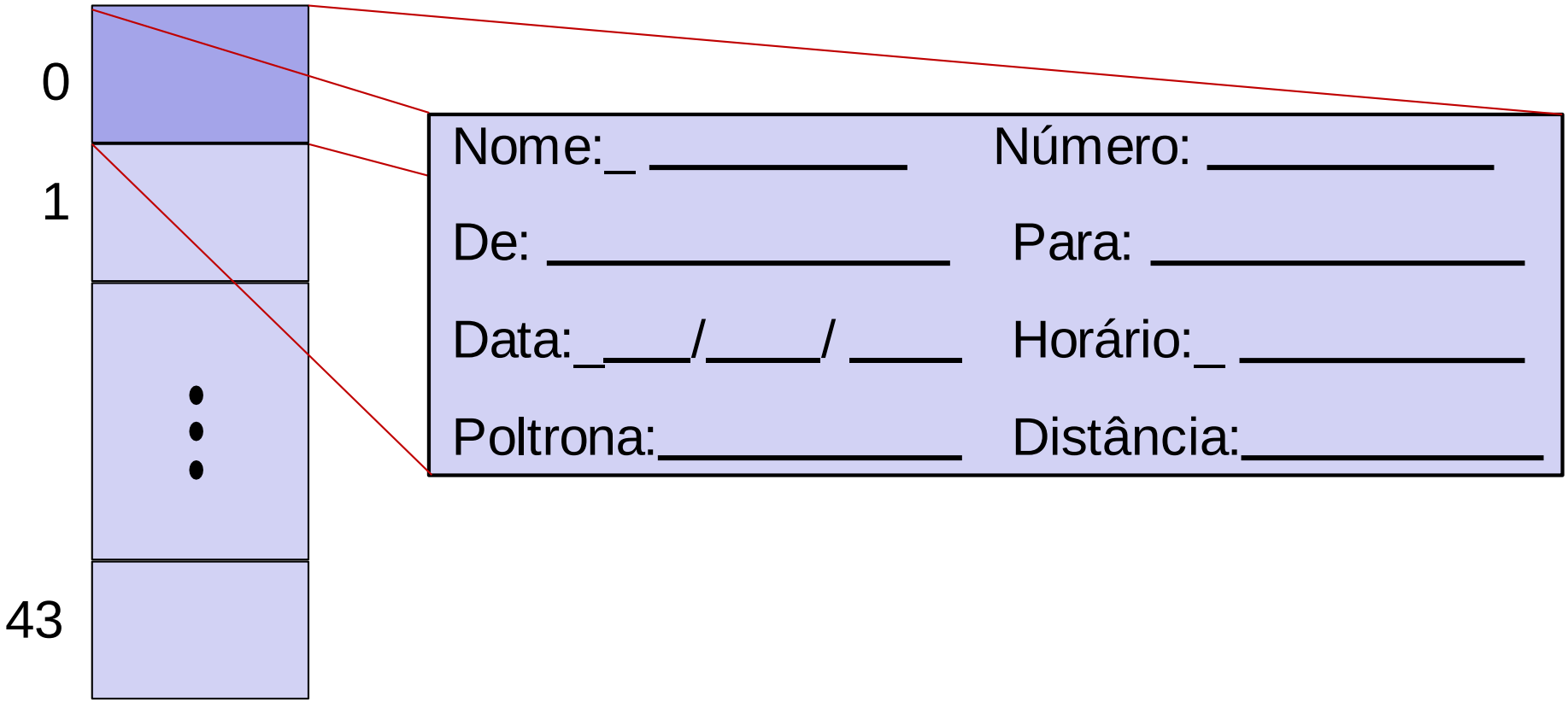
# Vetores de estruturas

- Suponha que deseja-se manter um registro de informações relativas a passagens rodoviárias de todos lugares (poltronas) de um ônibus.

- Pode-se utilizar uma estrutura referente a cada poltrona (`passagem`) e para agrupar todas elas utiliza-se um vetor de estruturas.

# Vetores de estruturas

- Um ônibus possui 44 lugares numerados de 0 a 43, passa por várias cidades e precisa armazenar as informações abaixo:



# Vetores de estruturas

- Definição da estrutura:

```
typedef struct
{
   char nome[50];
   int numero;
   char origem[20];
   char destino[20];
   char data[8];
   char horario[5];
   int poltrona;
   float distancia;
} Passagem;
```

- Declaração do vetor de estruturas:

```
Passagem dadosPassagem[44];
```

# Vetores de estruturas

- Declaração do vetor de estruturas:

```
Passagem dadosPassagem[44];
```

- Observe que cada posição do vetor de estruturas pode ser vista como uma variável do tipo Passagem.

```
Passagem p, dadosPassagem[44];
p = dadosPassagem[1];
```

# Vetores de estruturas

- Em cada posição do vetor, os campos de Passagem podem ser acessados:

```
Passagem dadosPassagem[44];

dadosPassagem[2].numero = 2;
scanf("%f", &dadosPassagem[2].distancia);
gets(dadosPassagem[2].origem);
gets(dadosPassagem[2].destino);
if(dadosPassagem[2].distancia > 50.0)
    puts(dadosPassagem[2].nome);
```

# Vetores de estruturas

- Em cada posição do vetor, os campos de Passagem podem ser acessados:

```
int i;
Passagem dadosPassagem[44];
for( i=0; i<44;  i++ ) {
   dadosPassagem[i].numero = i;
   scanf("%f", &dadosPassagem[i].distancia);
   gets(dadosPassagem[i].origem);
   gets(dadosPassagem[i].destino);
   if(dadosPassagem[i].distancia > 50.0)
      puts(dadosPassagem[i].nome);
}
```

# Inicialização de estruturas

```
typedef struct
{
   int codigo;
   char descricao[120];
} produto;
```

- Pode-se inicializar um vetor de estruturas no momento de sua criação como o código abaixo:

```
produto estoque[6] =  {{235, "Teclado USB"},
                       {245},
                       {515, "Memoria DDR"} };
```

# Inicialização de estruturas

```
typedef struct
{
  int codigo;
  char descricao[120];
} produto;
```

- Se o número de elementos inicializados for menor que os alocados em memória, o restante do vetor ficará “zerado” (se numérico) ou vazio (se *strings*).

```
produto estoque[6] =  {{235, "Teclado USB"},
                       {245},
                       {515, "Memoria DDR"} };
```

- No exemplo, o índice 1 do vetor terá descrição vazia e os índices 3, 4 e 5 terão código 0 e descrição vazia.

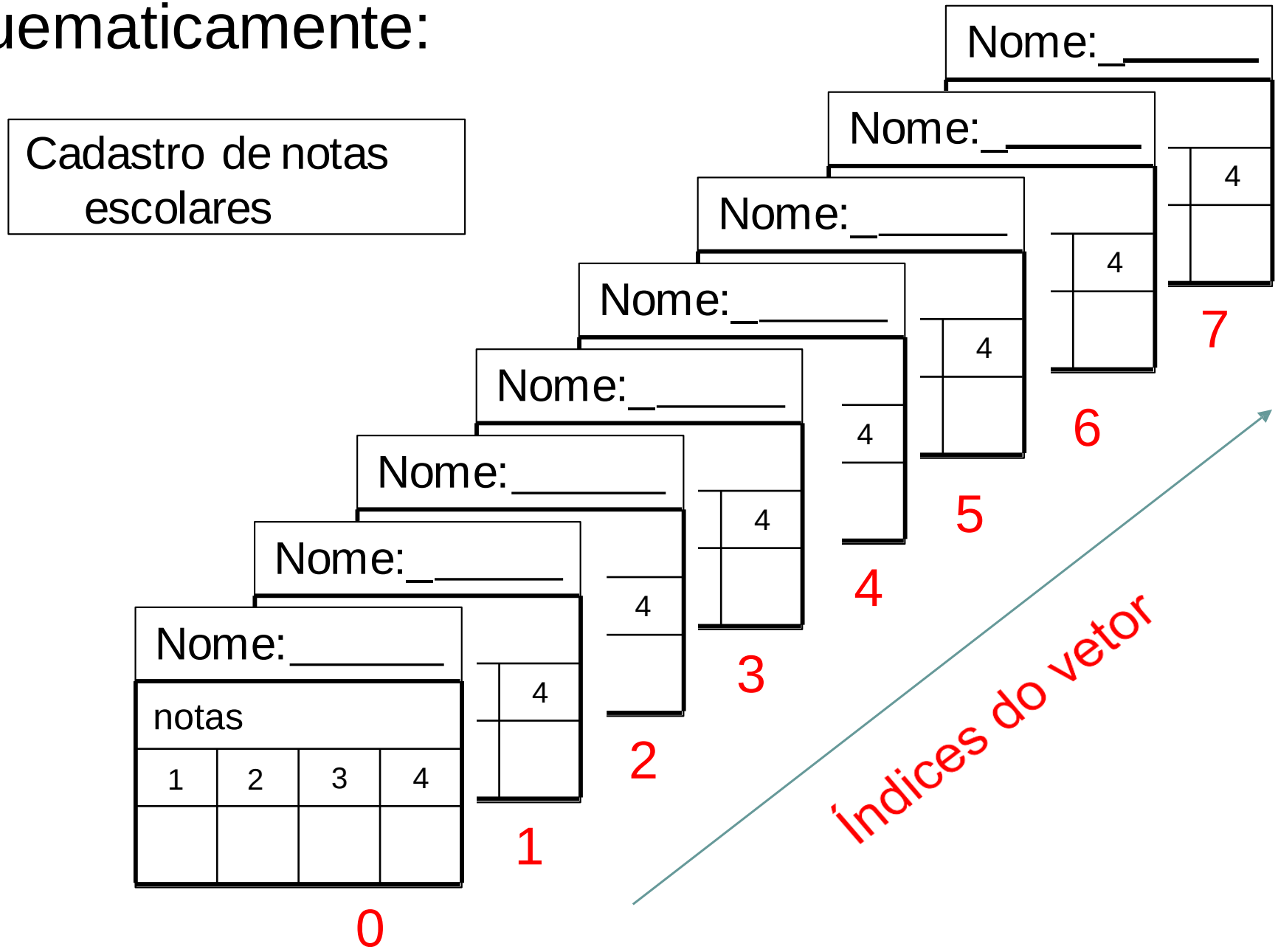
# Vetores de estruturas

- Outro exemplo mais completo....

- Considere que você está fazendo um programa que leia o nome e as 4 notas escolares de 8 alunos.

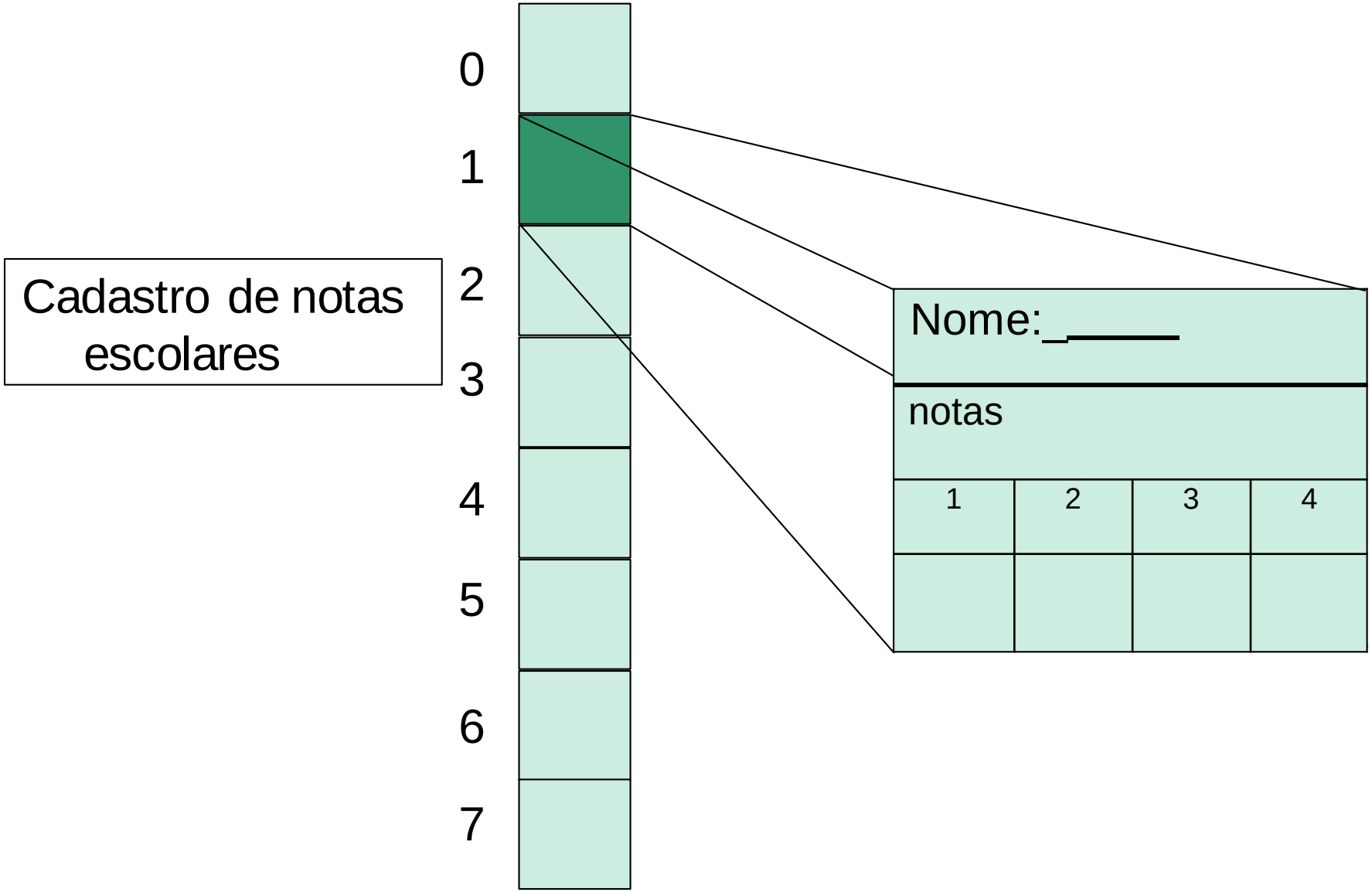
# Vetores de estruturas

- Esquematicamente:



# Vetores de estruturas

- Visão esquemática do vetor de estruturas:



# Vetores de estruturas

- Definição da estrutura e do vetor de 8 alunos.

```
typedef struct
{
   char nome[40];
   float notas[4];
} Aluno;

int main()
{
    Aluno notasAlunos[8];
    ...
```

# Leitura de vetor de estruturas

- Procedimento para ler os dados dos alunos:

```
void le_vetor_alunos(Aluno A[8])
{
  int i,j;
   for (i = 0; i < 8; i  = i + 1)
   {
     gets(A[i].nome);
     for (j = 0; j < 4;  j = j + 1)
     {
       scanf("%f",&A[i].notas[j]);
     }
   }
 }
```

# Leitura de vetor de estruturas

- Procedimento para ler os dados dos alunos:

```
void le_vetor_alunos(Aluno A[8])
{
   int i,j;
   for (i = 0; i < 8; i = i + 1)
   {
      gets(A[i].nome);
      for (j = 0; j < 4;  j = j + 1)
      {
         scanf("%f",&A[i].notas[j]);
      }
   }
}
```

Assim como ocorre com tipos básicos:

- Alterações feitas em uma **variável** passada por parâmetro **não têm efeito** fora da função;

- Alterações feitas em um **vetor são mantidas** fora da função.

# Leitura de vetor de estruturas

- Procedimento para ler os dados dos alunos:

Laço para ler os oito alunos

```
void le_vetor_alunos(Aluno A[8])
{
  int i,j;
  for (i = 0; i < 8; i = i + 1)
   {
     gets(A[i].nome);
     for (j = 0; j < 4;  j = j + 1)
     {
       scanf("%f",&A[i].notas[j]);
     }
   }
}
```

# Leitura de vetor de estruturas

- Procedimento para ler os dados dos alunos:

```
void le_vetor_alunos(Aluno  A[8])
{
  int i,j;
  for (i = 0; i < 8; i  = i + 1)
  {
     gets(A[i].nome);
     for (j = 0; j < 4; j = j + 1)
     {
        Scanf ("%f",&A[i].notas[j]);
     }
  }
}
```

gets é usado para ler o nome de cada aluno

# Leitura de vetor de estruturas

- Procedimento para ler os dados dos alunos:

```
void le_vetor_alunos(Aluno  A[8])
{
  int i,j;
  for (i = 0; i < 8; i  = i + 1)
  {
    gets(A[i].nome);
    for (j = 0; j < 4;  j = j + 1)
    {
      scanf("%f",&A[i].notas[j]);
    }
  }
}
```

Para cada aluno, um laço para ler as suas quatro notas

# Leitura de vetor de estruturas

- Procedimento para ler os dados dos alunos:

```
void le_vetor_alunos(Aluno A[8])
{
  int i,j;
   for (i = 0; i < 8; i  = i + 1)
   {
     gets(A[i].nome);
     for (j = 0; j < 4;  j = j + 1)
     {
       scanf("%f",&A[i].notas[j]);
     }
   }
}
```

Índice i do vetor de alunos

Índice j do vetor de notas

# Impressão de vetor de estruturas

- O processo de escrita de um vetor de estrutura é similar aos modos de escrita anteriores já vistos.

```
void imprime_vetor_alunos(Aluno A[8])
{
  int i,j;
  for (i = 0; i < 8; i = i + 1)
  {
    puts(A[i].nome);
    for (j = 0; j < 4; j = j + 1)
    {
      printf(" %f", A[i].notas[j]);
    }
  }
}
```

# Programa completo

```c
#include <stdio.h>

typedef struct
{
  char nome[40];
  float notas[4];
} Aluno;

void leVetorAlunos(Aluno a[], int n)
{
  int i, j;
  for( i = 0; i < n; i++)
  {
    printf("Informe o nome do aluno");
    gets(a[i].nome);
    printf("Informe as 4 notas ");
    for( j = 0; j < 4; j++)
      scanf("%f%*c", &a[i].notas[j]);
  }
}
```

```c
void imprimeVetorAlunos(Aluno a[], int n)
{
  int i, j;
  printf("\nAlunos:");
  for( i = 0; i < n; i++)
  {
    printf("\n%20s ", a[i].nome);
    for( j = 0; j < 4; j++)
      printf("%.2f ", a[i].notas[j]);
  }
}

int main()
{
  Aluno turma[35];

  leVetorAlunos(turma, 35);
  imprimeVetorAlunos(turma, 35);

  return 0;
}
```

# Exemplo 2

```c
#include <stdio.h>
#define TAM 4

typedef struct
{
  int codigo;
  char descricao[120];
} Produto;

int main()
{
  Produto estoque[TAM];
  leProdutos(estoque);
  imprimeProdutos(estoque);
  return 0;
}
```

```c
void leProdutos(Produto p[TAM])
{
  int i;
  printf("Produtos armazenados:\n");
  for( i = 0; i < TAM; i++)
  {
    printf("Produto %d: ", i);
    printf("Codigo: ");
    scanf("%d%*c", &p[i].codigo);
    printf("Descricao: ");
    gets(p[i].descricao);
  }
}
void imprimeProdutos(Produto p[TAM])
{
  int i;
  for( i = 0; i < TAM; i++)
    printf("%3d - %s\n",
      p[i].codigo, p[i].descricao);
}
```

# Exercícios

5) a) Crie uma estrutura chamada `ponto` contendo apenas as coordenadas `x` e `y` (inteiros) do ponto.

b) Faça uma função que receba dois pontos por parâmetro e retorne a distância entre eles.

c) Faça um programa que declare 2 pontos, leia as coordenadas `x` e `y` de cada um e chame a função criada para calcular a distância entre eles. Apresente no final a distância entre os dois pontos.

6) Utilizando a estrutura `ponto` definida no exercício 5, faça uma função que receba 2 pontos e retorne o ponto mais próximo da origem. Em seguida, modifique o programa principal para ler 4 pontos e imprimir apenas as coordenadas do ponto mais próximo da origem.

# Exercícios

7) Utilizando a estrutura `medidas` definida no exercício 2, faça um programa que leia a altura e o peso de 6 pessoas e imprima a média da altura e a média do peso do grupo.

8) Faça um programa que receba três nomes (de no máximo 100 caracteres cada) e as idades das respectivas pessoas em um vetor de estruturas. Após o recebimento, listar os três nomes armazenados neste vetor por ordem crescente de idades.

9) Faça um programa que armazene informações de restaurantes. Cada restaurante é identificado pelo nome, o tipo de comida (brasileira, chinesa, francesa, italiana, japonesa, etc.) e uma nota para a cozinha (entre 0 e 5). O programa deverá ler todas as informações e imprimir a lista de todos os restaurantes e, ao final, o tipo de cozinha do restaurante com maior nota.

# Exercícios

10) Faça um programa que permita o cadastro de veículos. A estrutura veículos deverá conter os campos placa, marca, modelo e ano. Faça um menu com as seguintes opções:

```
Menu:

1   - Ler as informações de um veículo

2   - Verificar se uma placa está no formato
    correto  (AAADDDD)

3 - Imprimir por ano

4 - Pesquisar veículo por placa

5 - Imprimir todos os veículos cadastrados
```

O programa deverá ter as seguintes características:

- No primeiro item, peça inicialmente o índice do vetor que deseja alterar.
- No terceiro item, peça o ano mínimo e máximo e imprima os veículos que estão nesse intervalo.
- Faça funções para realizar as operações de cada um dos itens do menu.

# Recursividade - Definição

- Uma função é dita recursiva quando chama a si própria, direta ou indiretamente.

- Propósito: expressar sua funcionalidade ou objetivo de maneira mais clara e concisa.

- Linguagens que permitem uma função chamar a si própria são ditas recursivas
  - A linguagem C é uma linguagem recursiva.

# Exemplo de Algoritmo recursivo

**Cálculo do valor de $2^n$ para um número n:**

Aprendemos que $2^n = 2 * 2 * 2 * \ldots * 2$.

# Exemplo de Algoritmo recursivo

Implementação iterativa muito simples:

```
int Potencia(int n)
{
  int Pot, i;
  Pot = 1;
  for (i = 1; i <= n; i++)
     Pot = Pot * 2;
  return (Pot);
}
```

# Exemplo de Algoritmo recursivo

Calcular o valor de $2^n$ como sendo $2 * 2^{n-1}$:

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

# Exemplo de Algoritmo recursivo

- Significa que, para cálculo do valor de $2^n$ para um determinado n, há necessidade de que se recorra aos cálculos anteriores.

Por exemplo: Quanto é o $2^4$?

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 =$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3$

$2^3 =$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3$

$2^3 = 2 * 2^2$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3$

$2^3 = 2 * 2^2$

$2^2 =$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3$

$2^3 = 2 * 2^2$

$2^2 = 2 * 2^1$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3$

$2^3 = 2 * 2^2$

$2^2 = 2 * 2^1$

$2^1 =$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3$

$2^3 = 2 * 2^2$

$2^2 = 2 * 2^1$

$2^1 = 2$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3$

$2^3 = 2 * 2^2$

$2^2 = 2 * 2^1 = 2 * 2 = 4$

$2^1 = 2$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3$

$2^3 = 2 * 2^2 = 2 * 4 = 8$

$2^2 = 2 * 2^1 = 2 * 2 = 4$

$2^1 = 2$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3 = 2 * 8 = 16$

$2^3 = 2 * 2^2 = 2 * 4 = 8$

$2^2 = 2 * 2^1 = 2 * 2 = 4$

$2^1 = 2$

# Exemplo de Algoritmo recursivo

$2^4$

$$2^n \begin{cases} 2, \text{ se } n = 1 \\ 2 * 2^{n-1}, \text{ se } n > 1 \end{cases}$$

$2^4 = 2 * 2^3 = 2 * 8 = 16$

$2^3 = 2 * 2^2 = 2 * 4 = 8$

$2^2 = 2 * 2^1 = 2 * 2 = 4$

$2^1 = 2$

Portanto, $2^4 = 16$

# Exemplo de Algoritmo recursivo

- Condição que define parada da recursão: chamada base da recursão ou de caso base

- Todo programa recursivo deve possuir uma condição de parada !

- Condição que define recursão chamada passo recursivo.

# Exemplo de Algoritmo recursivo

$$2^n \begin{cases} 2, \text{ se } n = 1 & \Leftarrow \text{ Base da recursão} \\ 2 * 2^{n-1}, \text{ se } n > 1 & \Leftarrow \text{ Passo recursivo} \end{cases}$$

# Exemplo de Algoritmo recursivo

```
int PotenciaRecursivo (int n)
{
    if  (n == 1)
        return (2);
    else
        return (2 * PotenciaRecursivo (n - 1));
}
```

Base da recursão

Passo recursivo

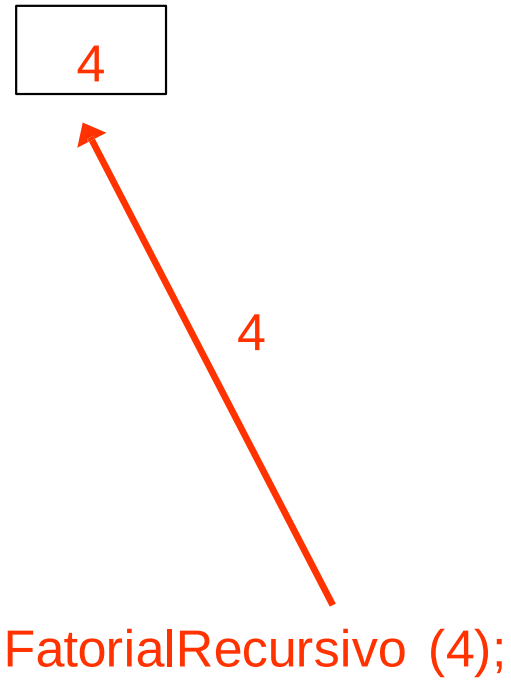
# Simulando o algoritmo

Mas o que acontece durante a execução de uma função recursiva?

Suponha que função chamada em algum ponto do código:

FatorialRecursivo (4);

# Simulando o algoritmo




```
int FatorialRecursivo (int n)
{
  if    (n == 1)
  {
     return 1;
  }
  else
  {
     return (n * FatorialRecursivo (n - 1));
  }
}
```

# Simulando o algoritmo

4

FatorialRecursivo (4)

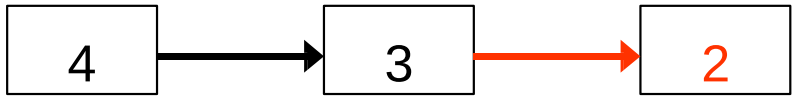
```
int FatorialRecursivo (4)
{
  if   (4 == 1)
  {
     return 1;
  }
  else
  {
     return (4 * FatorialRecursivo (4 - 1));
  }
}
```

# Simulando o algoritmo

FatorialRecursivo (3)

```
int FatorialRecursivo (3)
{
  if    (3 == 1)
  {
     return 1;
  }
  else
  {
     return (3 * FatorialRecursivo (3 - 1));
  }
}
```

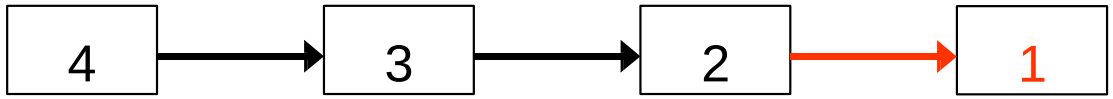
# Simulando o algoritmo




FatorialRecursivo (2)

```
int FatorialRecursivo (2)
{
  if    (2 == 1)
  {
     return 1;
  }
  else
  {
     return (2 * FatorialRecursivo (2 - 1));
  }
}
```

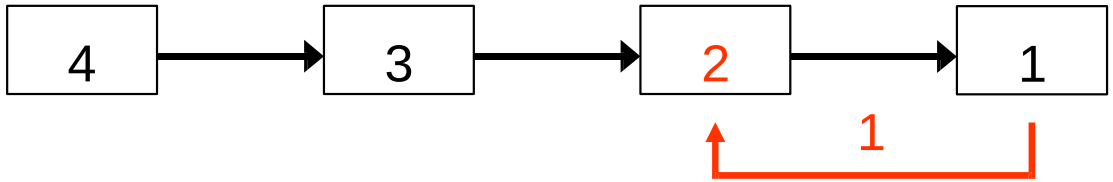
# Simulando o algoritmo




FatorialRecursivo (1)

```
int FatorialRecursivo (1)
{
  if   (1 == 1)
  {
     return 1;
  }
  else
  {
     return (n * FatorialRecursivo (n - 1));
  }
}
```

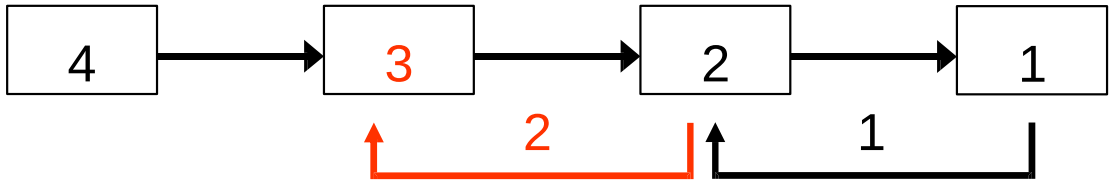
# Simulando o algoritmo




FatorialRecursivo (2)

```
int FatorialRecursivo (2)
{
  if    (2 == 1)
  {
     return 1;
  }
  else
  { // return (2 * FatorialRecursivo (2 - 1));
       return (2 * 1));
  }
}
```

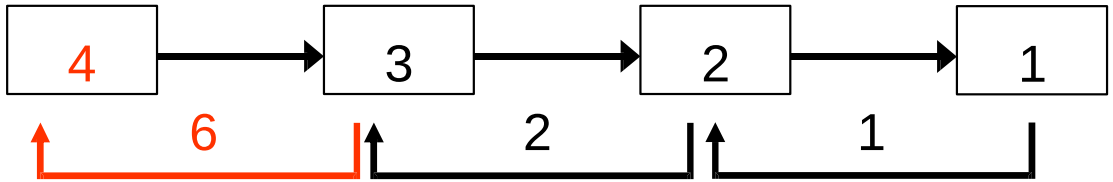
# Simulando o algoritmo




FatorialRecursivo (3)

```
int FatorialRecursivo (3)
{
  if    (3 == 1)
  {
     return 1;
  }
  else
  { // return (3 * FatorialRecursivo (3 - 1));
       return (3 * 2));
  }
}
```

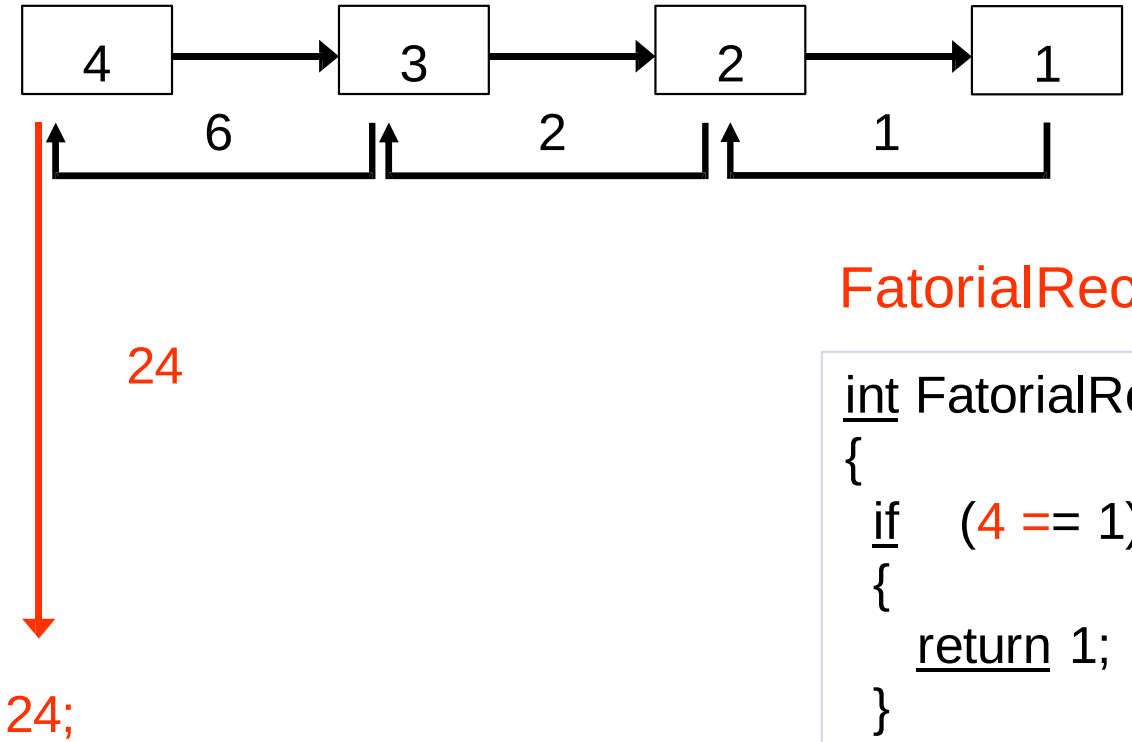
# Simulando o algoritmo




FatorialRecursivo (4)

```
int FatorialRecursivo (4)
{
  if    (4 == 1)
  {
     return 1;
  }
  else
  { // retorne (4 * FatorialRecursivo (4 - 1));
       return (4 * 6));
  }
}
```

# Simulando o algoritmo




FatorialRecursivo (4)

```
int FatorialRecursivo (4)
{
  if    (4 == 1)
  {
     return 1;
  }
  else
  { // return (4 * FatorialRecursivo (4 - 1));
       return (4 * 6));
  }
}
```

# Exercício

Escreva um código recursivo para calcular o máximo divisor comum, MDC, de dois números inteiros positivos N e M da seguinte maneira:

$$\mathbf{mdc}(n, m) = \begin{cases} m, & \text{se} \quad n \geq m \ \text{ e } \ n \ \mathbf{mod} \ m = 0 \\ mdc(m, n), & \text{se} \quad n < m \\ mdc(m, n \ \mathbf{mod} \ m), & \text{caso contrário} \end{cases}$$

# Exercício

$$\boxed{\mathbf{mdc}(n, m)} = \begin{cases} m, & \text{se} \quad n \geq m \text{ e } n \textbf{ mod } m = 0 \\ mdc(m, n), & \text{se} \quad n < m \\ mdc(m, n \textbf{ mod } m), & \text{caso contrário} \end{cases}$$

```
int MDC (int n, int m)
{

}
```

# Exercício

$$\mathbf{mdc}(n, m) = \begin{cases} m, & \text{se } n \geq m \text{ e } n \bmod m = 0 \\ mdc(m, n), & \text{se } n < m \\ mdc(m, n \bmod m), & \text{caso contrário} \end{cases}$$

```
int MDC (int n, int m)
{
  if ( (n>=m) && ((n%m)==0) )
   return(m);



}
```

# Exercício

$$\mathbf{mdc}(n, m) = \begin{cases} m, & \text{se} \quad n \geq m \text{ e } n \bmod m = 0 \\ mdc(m, n), & \text{se} \quad n < m \\ mdc(m, n \bmod m), & \text{caso contrário} \end{cases}$$

```
int MDC (int n, int m)
{
  if ( (n>=m) && ((n%m)==0) )
   return(m);
  else
  {



  }
}
```

# Exercício

$$\mathbf{mdc}(n, m) = \begin{cases} m, & \text{se} \quad n \geq m \text{ e } n \bmod m = 0 \\ \boxed{mdc(m, n), \quad \text{se} \quad n < m} \\ mdc(m, n \bmod m), & \text{caso contrário} \end{cases}$$

```
int MDC (int n, int m)
{
  if ( (n>=m) && ((n%m)==0) )
   return(m);
  else
  {
    if (n<m)
      return (MDC(m,n));



}
}
```

# Exercício

$$\mathbf{mdc}(n, m) = \begin{cases} m, & \text{se} \quad n \geq m \text{ e } n \bmod m = 0 \\ mdc(m, n), & \text{se} \quad n < m \\ \boxed{mdc(m, n \bmod m), \quad \text{caso contrário}} \end{cases}$$

```
int MDC (int n, int m)
{
  if ( (n>=m) && ((n%m)==0) )
   return(m);
  else
  {
    if (n<m)
      return (MDC(m,n));
    else
      return (MDC(m,n%m));
  }
}
```

# Exercícios

1) Escreva uma função recursiva para calcular o N-ésimo número da sequência de Fibonacci. A sequência é dada por:

$$F(n) = \begin{cases} 0, & \text{se } n = 0; \\ 1, & \text{se } n = 1; \\ F(n-1) + F(n-2) & \text{outros casos.} \end{cases}$$

2) Seja S um vetor de inteiros. Descreva funções recursivas para calcular:

   a) o elemento máximo de S;
   b) a soma dos elementos de S;
   c) média aritmética dos elementos de S.

3) Escreva um procedimento recursivo que imprima na tela os números ímpares de 1 à um número fornecido pelo usuário.

# Exercícios

4) Escreva um procedimento recursivo, ImprimeSerie (i, j, k), que imprime na tela a série de valores do intervalo [i,j], com incremento k (i, j e k são inteiros).

5) Escreva uma função recursiva int SomaSerie (i, j, k), que devolva a soma da série de valores do intervalo [i,j], com incremento k (i, j e k são inteiros).

6) Faça uma função recursiva que calcule o valor da série S descrita a seguir para um valor n > 0 a ser fornecido como parâmetro para a mesma:
$S = 1 + 1/2! + 1/3! + \ldots 1/n!$
OBS: A função fatorial também deve ser recursiva.

# Debugger (Depurador)

- O *debugger* ou depurador é uma funcionalidade do Code::Blocks e de outras IDEs que permite a verificação passo-a-passo da execução de um algoritmo.

- Impressões, leituras de dados, variáveis, vetores, strings etc. podem ter seus valores verificados a cada execução de uma nova instrução.

- Assim, possíveis falhas no algoritmo podem ser detectadas facilmente.

# Debugger (Depurador)

- O *debugger* também serve como uma importante ferramenta para o aprendizado de algoritmos, permitindo que seja feito um “teste de mesa” durante a execução do próprio algoritmo.

- Através do processo de *debugging*, o fluxo de execução de um código pode ser verificado continuamente, por meio da checagem do resultado de cada teste condicional, de cada repetição, de cada operação lógica e matemática etc..

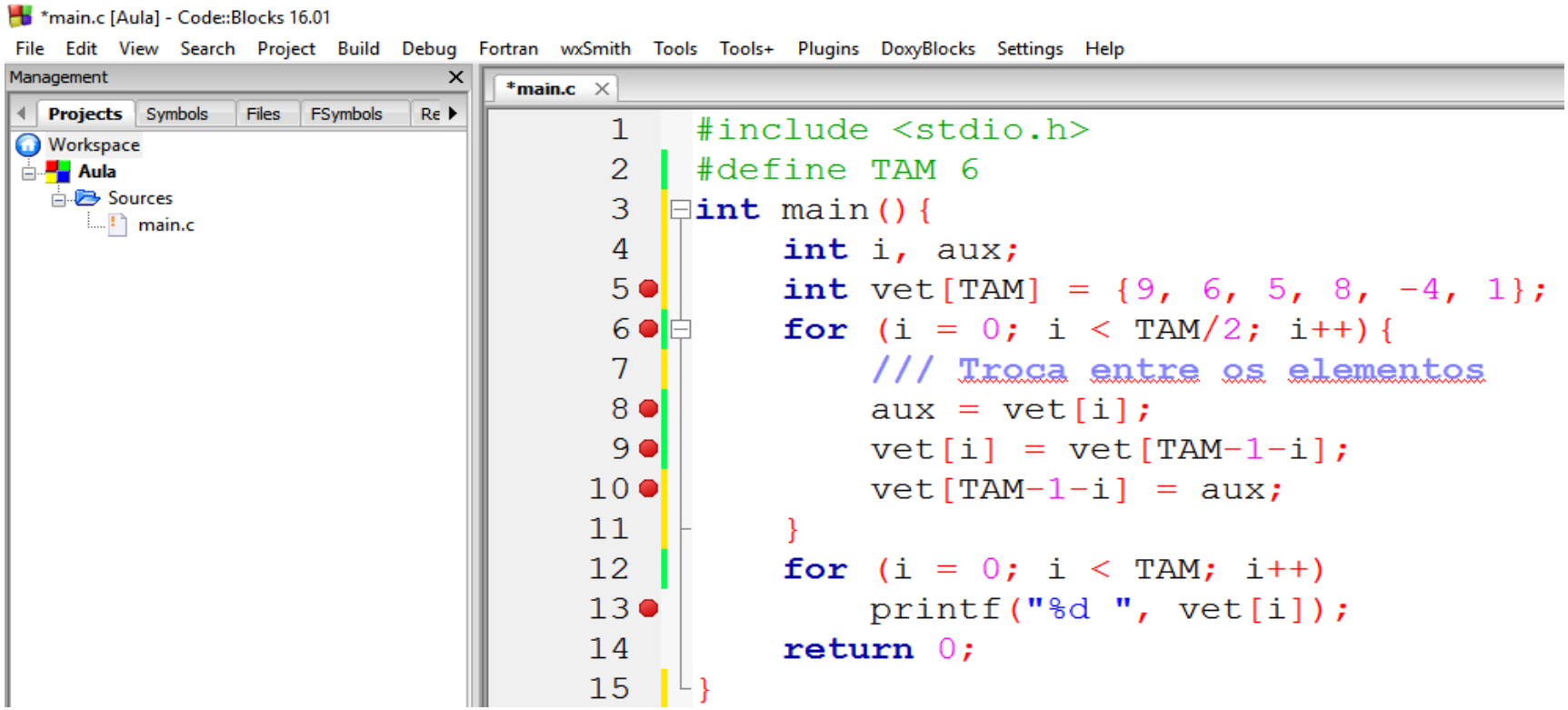
# Como utilizar?

- O algoritmo abaixo será utilizado para a explicação do funcionamento do *debugger* do Code::Blocks.
- O algoritmo inverte a ordem dos elementos de um vetor de inteiros.

```
1   #include <stdio.h>
2   #define TAM 6
3   int main(){
4       int i, aux;
5       int vet[TAM] = {9, 6, 5, 8, -4, 1};
6       for (i = 0; i < TAM/2; i++){
7           /// Troca entre os elementos
8           aux = vet[i];
9           vet[i] = vet[TAM-1-i];
10          vet[TAM-1-i] = aux;
11      }
12      for (i = 0; i < TAM; i++)
13          printf("%d ", vet[i]);
14      return 0;
15  }
```

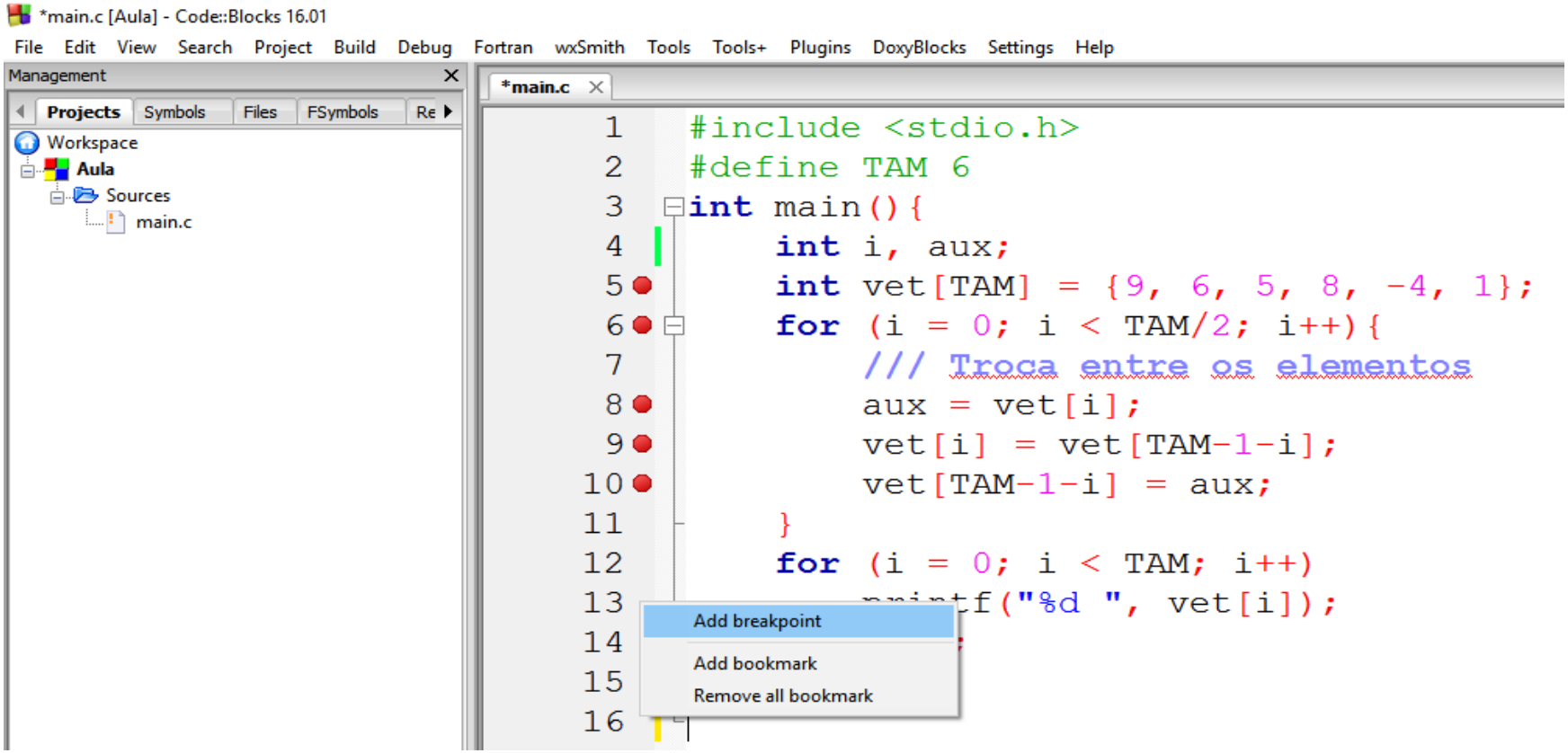
# Como utilizar?

- Antes de dar início ao *debugging*, é necessário colocar *breakpoints* (pontos de parada) nas linhas cujas execuções devem ser verificadas.
- Basta clicar ao lado direito do número de cada linha desejada. Os hexágonos em vermelho indicam a criação de um *breakpoint*.



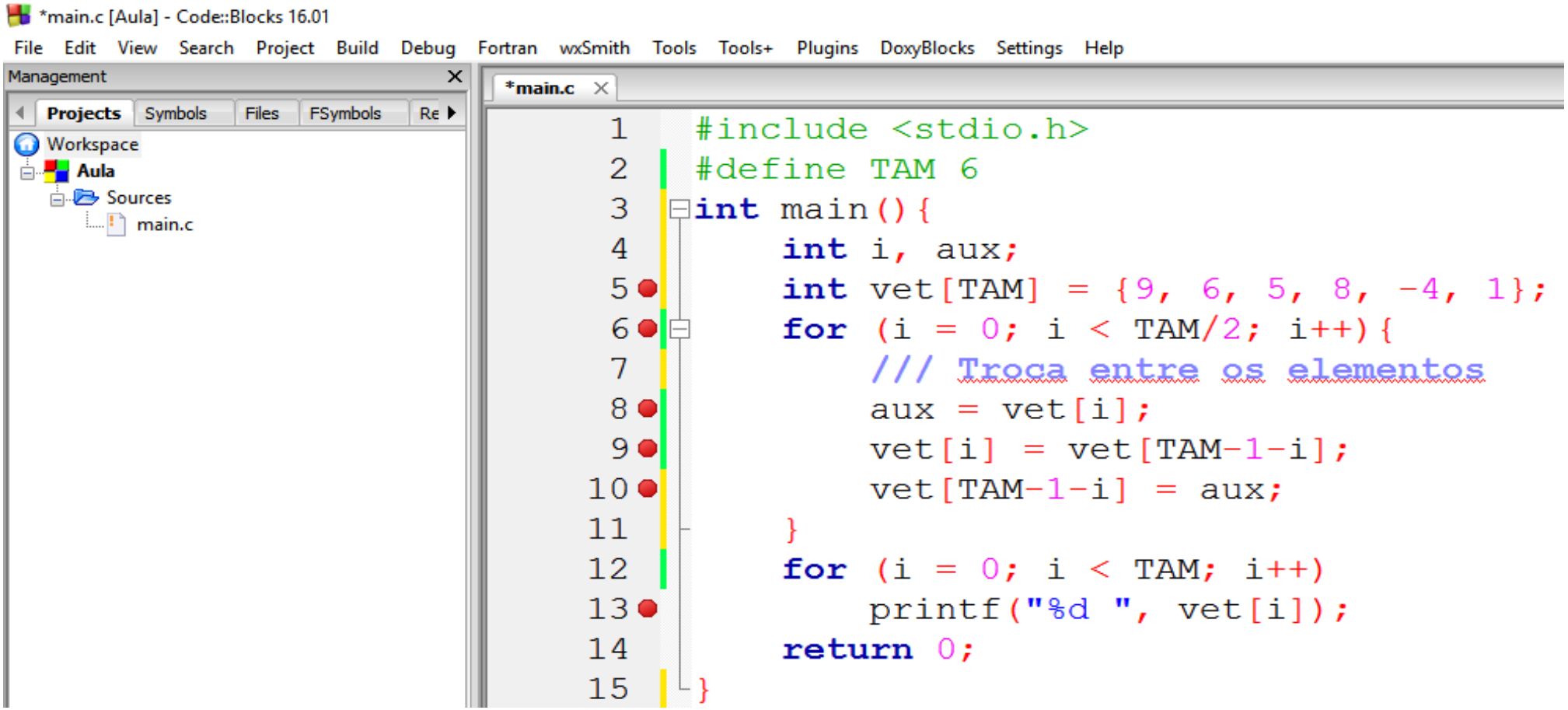
# Como utilizar?

- Outras formas de se adicionar um *breakpoint*:
  - Clicar com o botão esquerdo do mouse ao lado direito do número de cada linha desejada e escolher a opção *Add breakpoint*;
  - Colocar o cursor na linha desejada e pressionar a tecla F5.



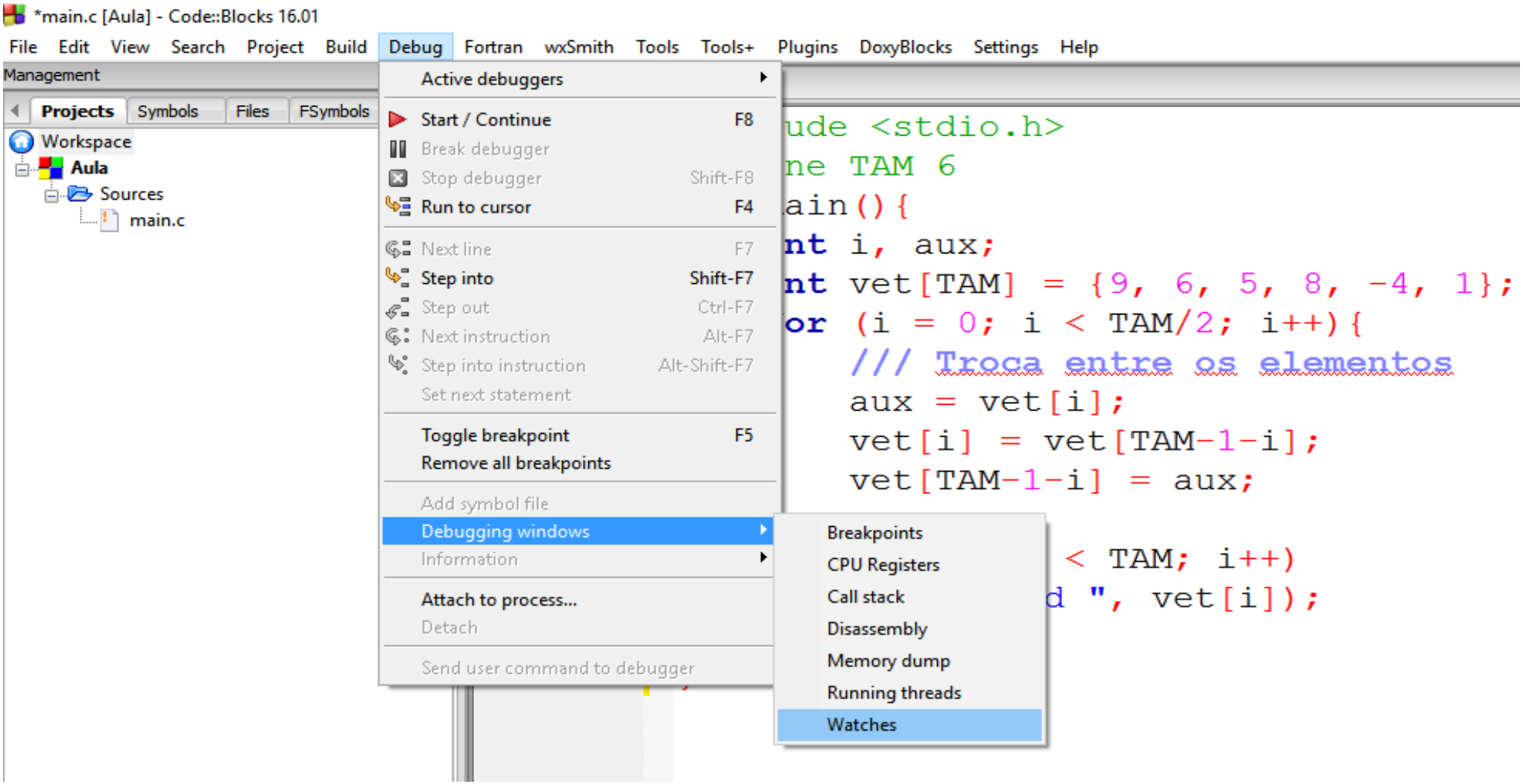
# Como utilizar?

- Com a adição dos *breakpoints*, nas linhas 5, 6, 8, 9, 10 e 13 do algoritmo abaixo, ocorrerá o passo-a-passo da execução.



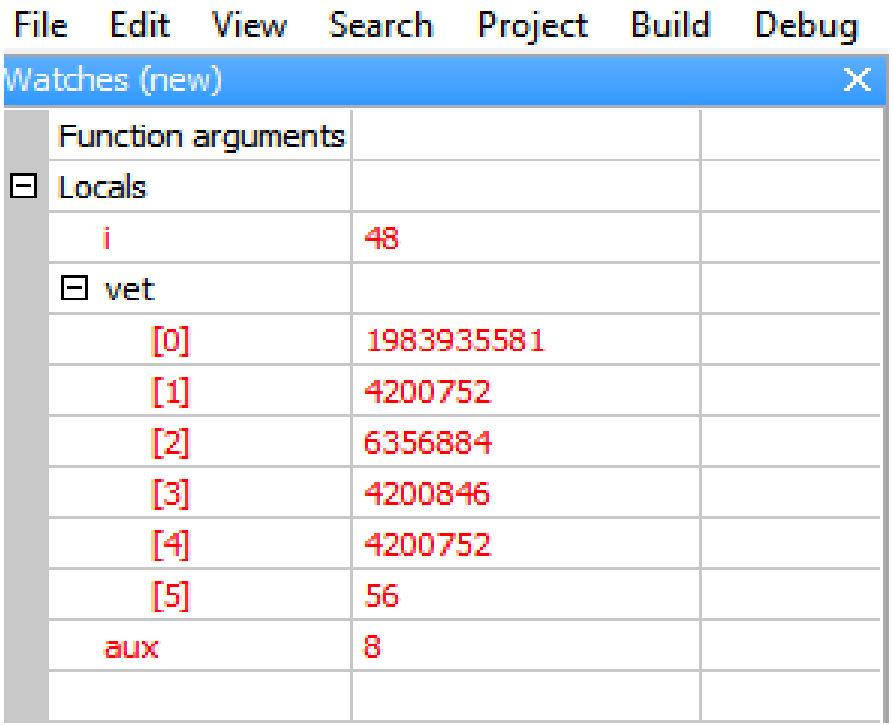
# Como utilizar?

- No menu *Debug*, acesse *Debugging windows* e clique em *Watches*.



# Como utilizar?

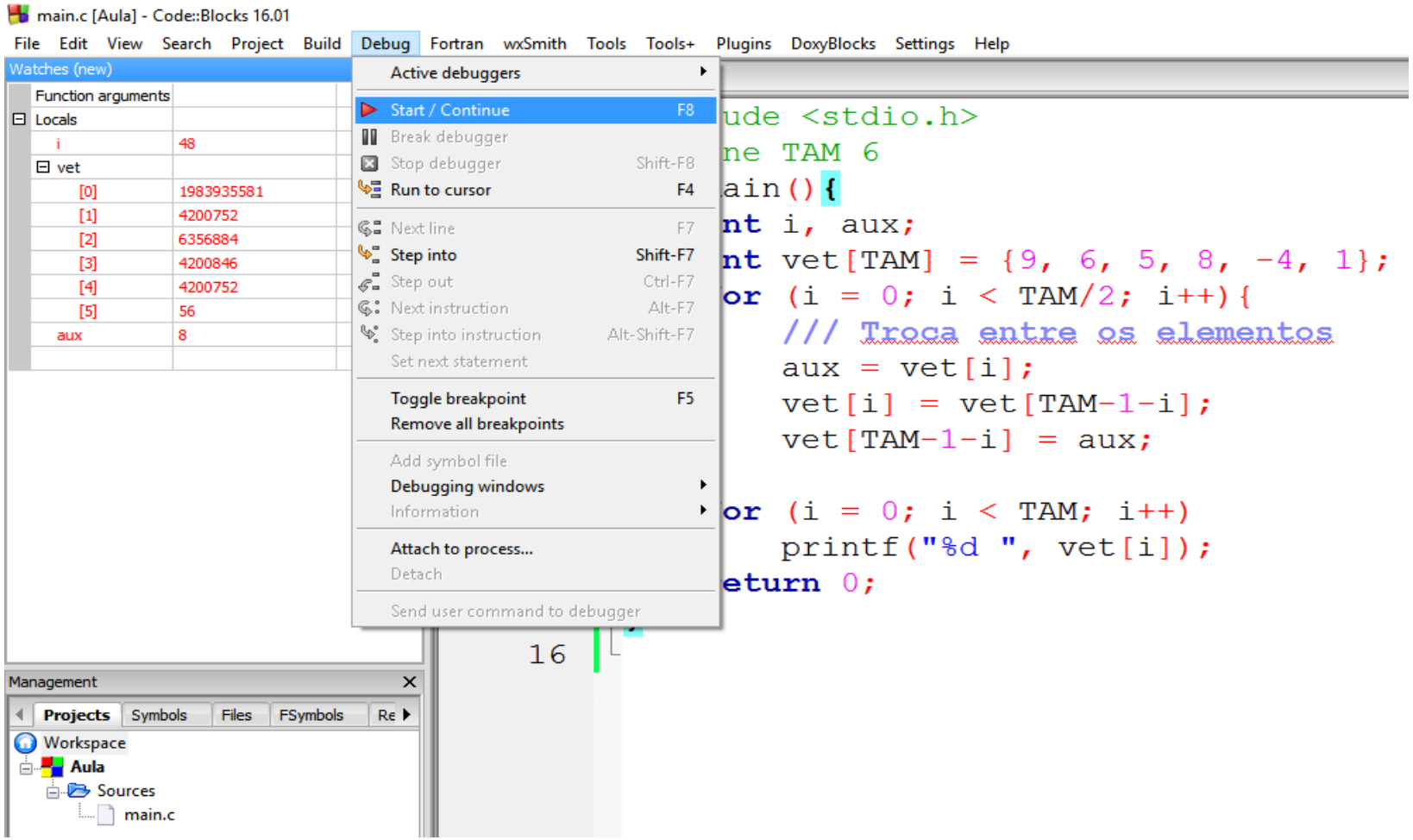
- A janela *Watches* será acionada. Nela é possível acompanhar o conteúdo das variáveis, vetores, strings etc., a cada passo da execução do algoritmo.




Inicialmente, as variáveis `i` e `aux` e o vetor `vet` contém lixo de memória.

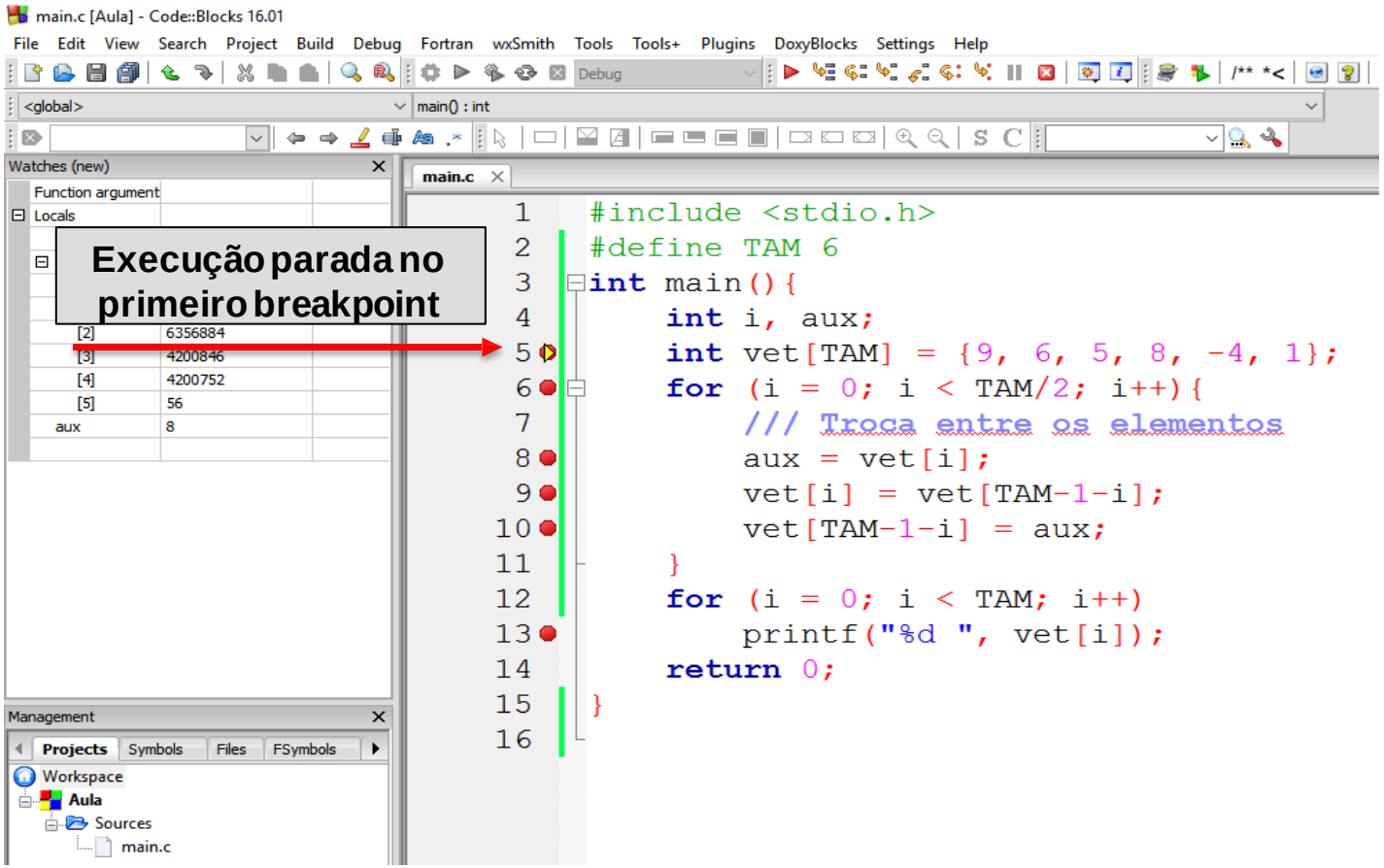
# Como utilizar?

- Para dar início ao *debugging*, acesse o menu *Debug* e, em seguida, escolha a opção *Start/Continue* ou pressione a tecla *F8.*



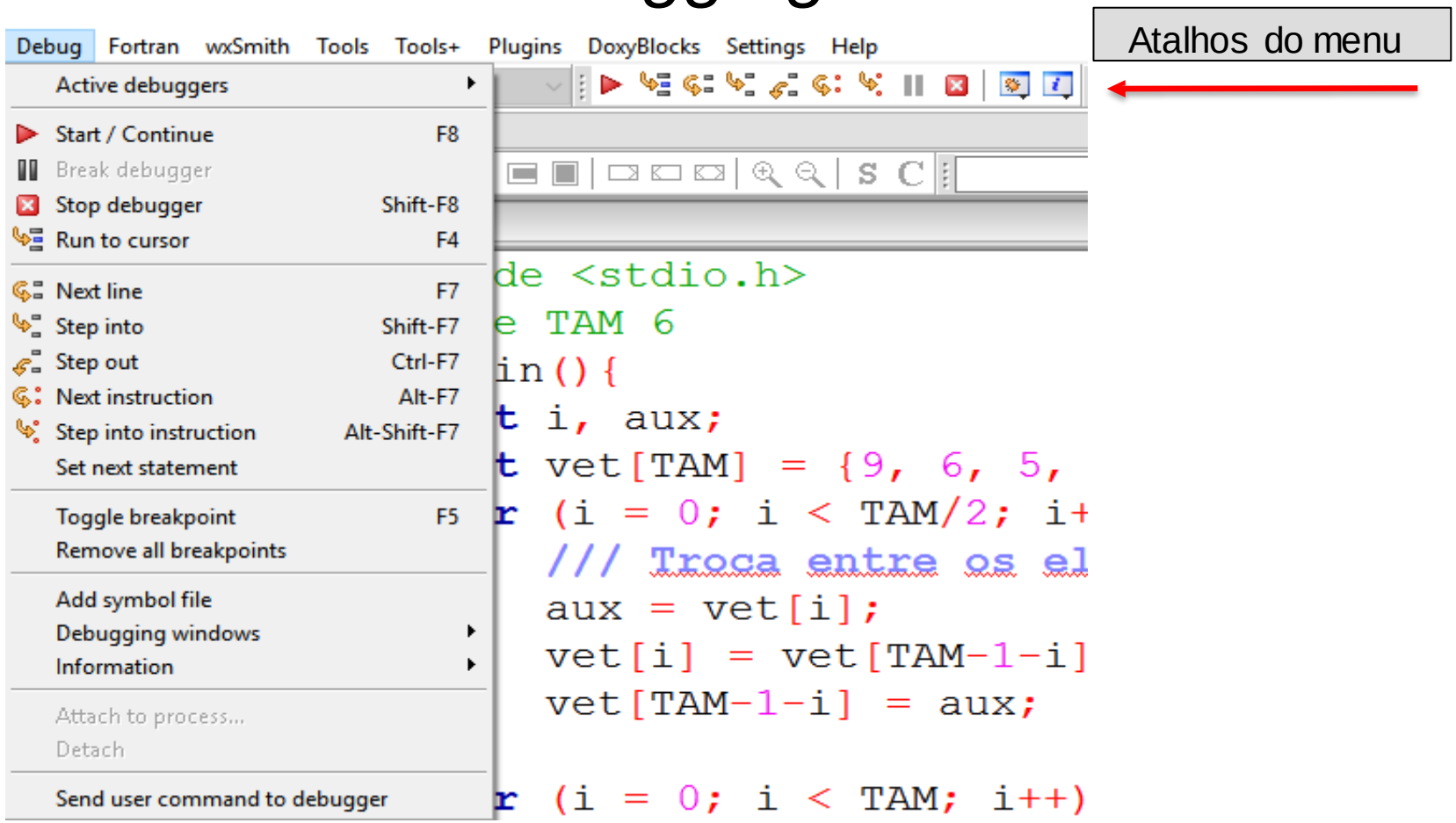
# Como utilizar?

- O algoritmo será executado normalmente até que seja encontrado um *breakpoint*. A seta em amarelo indica **a linha que será executada** (linha 5).



# Controle da execução

- O controle do fluxo de execução pode ser feito através de alguns comandos dispostos no menu *Debug*, destacado abaixo, que é exibido a partir do início do *debugging*.



# Controle da execução

***Next instruction* (Alt-F7):** a próxima instrução é executada.

***Next line* (F7):** a execução é desviada para a próxima linha.

***Run to cursor* (F4):** a execução é desviada para a linha na qual se encontra o cursor.

***Start/Continue* (F8):** o processo de *debugging* é iniciado (*start*). O *debugging* é executado passo-a-passo (*continue*).

# Controle da execução

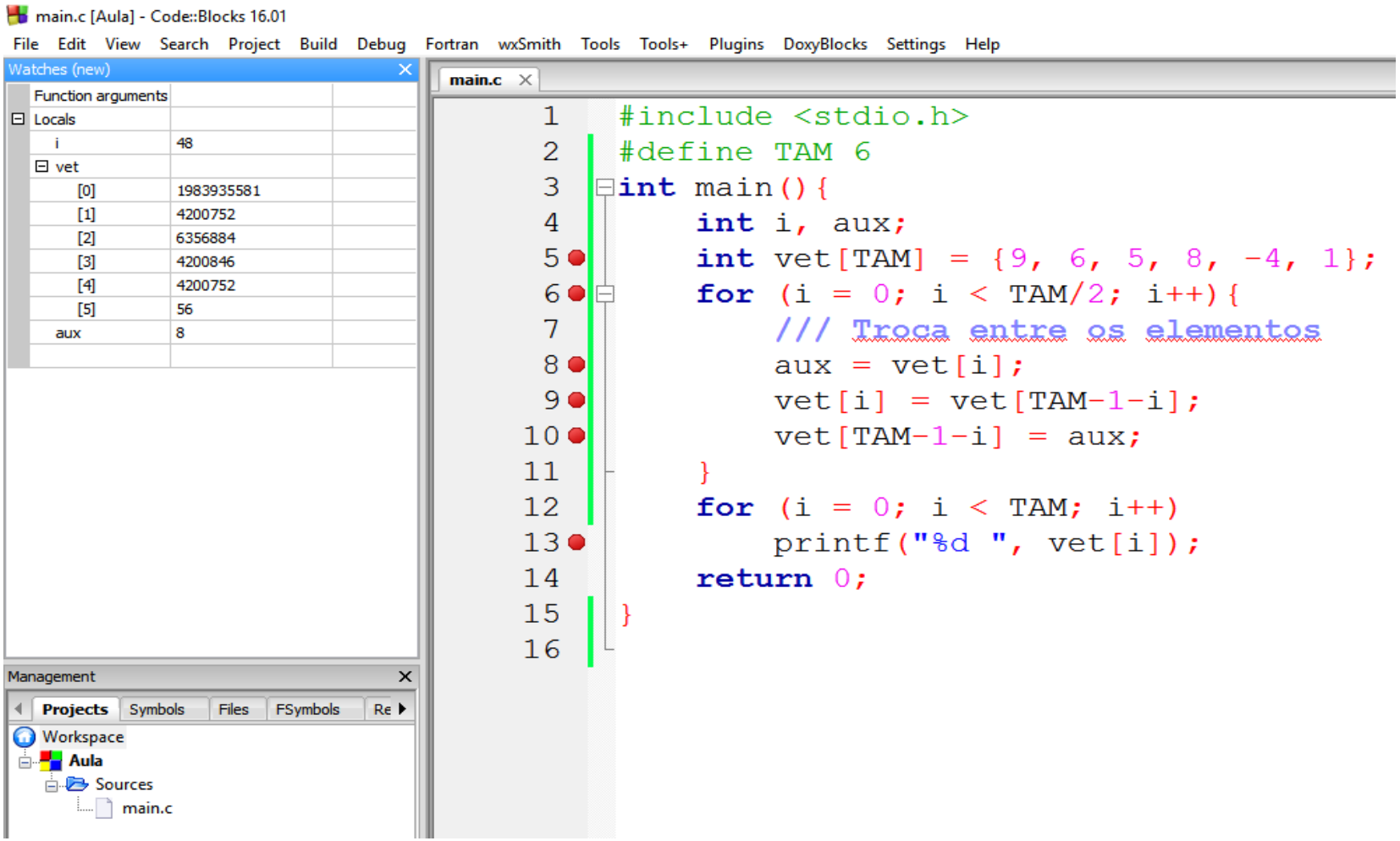
***Step into* (Shitf-F7):** se a instrução a ser executada for uma chamada de função, a execução é desviada para a função.

***Step out* (Ctrl-F7):** a função em execução é interrompida. O fluxo retorna para a instrução seguinte à chamada da função.

***Stop debugger* (Shift-F8):** O processo de *debugging* é finalizado.

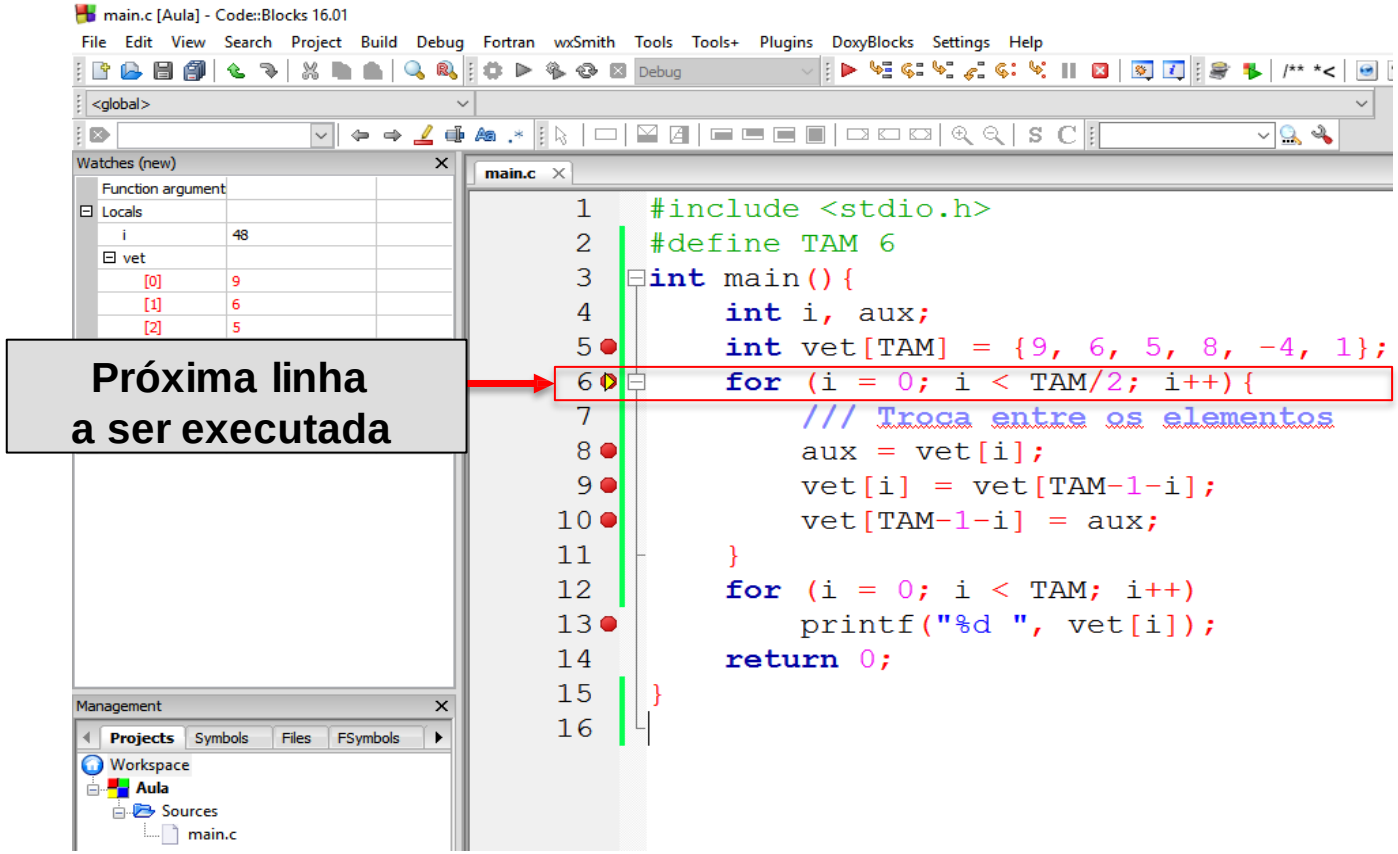
# Execução

- A seguir, o algoritmo descrito anteriormente será executado passo-a-passo por meio do comando *Next line* (F7)



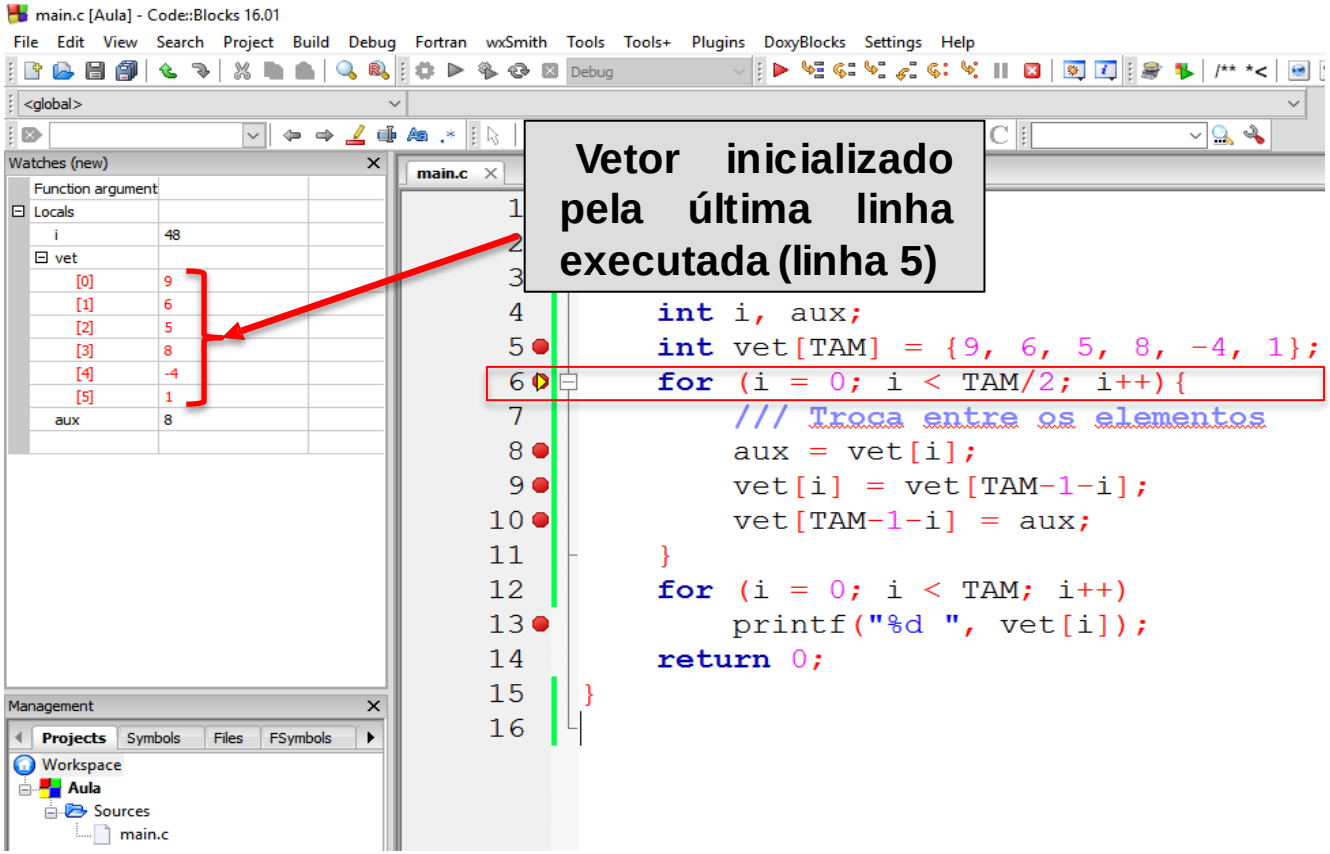
# Execução

- As linhas em destaque indicam as instruções que serão executadas na sequência.
- Assim, em *Watches* e na tela de impressão os conteúdos exibidos se referem à última execução do algoritmo realizada pelo *debugger*.



# Execução

- As linhas em destaque indicam as instruções que serão executadas na sequência.
- Assim, em *Watches* e na tela de impressão os conteúdos exibidos se referem à última execução do algoritmo realizada pelo *debugger*.



# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 48 |
| vet | |
| [0] | 1983935581 |
| [1] | 42 |
| [2] | 63 |
| [3] | 4200848 |
| [4] | 4200752 |
| [5] | 56 |
| aux | 8 |

Início da execução

main.c

```
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 48 |
| vet | |
| [0] | 1983935581 |
| [1] | 4200752 |
| [2] | 6356884 |
| [3] | 4200846 |
| [4] | 4200752 |
| [5] | 56 |
| aux | 8 |

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

main.c

```
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

Breakpoints

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 48 |
| vet | |
| [0] | 1983935581 |
| [1] | 4200752 |
| [2] | 6356884 |
| [3] | 4200846 |
| [4] | 4200752 |
| [5] | 56 |
| aux | 8 |

Variáveis e vetor contendo lixo de memória

```
2   #define TAM 6
3   int main(){
4       int i, aux;
5       int vet[TAM] = {9, 6, 5, 8, -4, 1};
6       for (i = 0; i < TAM/2; i++){
7           /// Troca entre os elementos
8           aux = vet[i];
9           vet[i] = vet[TAM-1-i];
10          vet[TAM-1-i] = aux;
11      }
12      for (i = 0; i < TAM; i++)
13          printf("%d ", vet[i]);
14      return 0;
15  }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 48 |
| vet | |
| [0] | 9 |
| [1] | 6 |
| [2] | 5 |
| [3] | 8 |
| [4] | -4 |
| [5] | 1 |
| aux | 8 |

Vetor inicializado

main.c

```
1   #include <stdio.h>
    #define TAM 6
3   int main(){
4       int i, aux;
5       int vet[TAM] = {9, 6, 5, 8, -4, 1};
6       for (i = 0; i < TAM/2; i++){
7           /// Troca entre os elementos
8           aux = vet[i];
9           vet[i] = vet[TAM-1-i];
10          vet[TAM-1-i] = aux;
11      }
12      for (i = 0; i < TAM; i++)
13          printf("%d ", vet[i]);
14      return 0;
15  }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 0 |
| vet | |
| [0] | 9 |
| [1] | 6 |
| [2] | 5 |
| [3] | 8 |
| [4] | -4 |
| [5] | 1 |
| aux | 8 |

Variável inicializada

main.c

```
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 0 |
| vet | |
| [0] | 9 |
| [1] | 6 |
| [2] | 5 |
| [3] | 8 |
| [4] | -4 |
| [5] | 1 |
| aux | 9 |

Início da inversão do primeiro elemento com o último

main.c

```c
1  #include <stdio.h>
2  #define TAM 6
   t main(){
       int i, aux;
       int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> | main() : int

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 0 |
| vet | |
| [0] | 1 |
| [1] | 6 |
| [2] | 5 |
| [3] | 8 |
| [4] | -4 |
| [5] | 1 |
| aux | 9 |

Continuação...

main.c

```c
#include <stdio.h>
#define TAM 6
int main(){
    int i, aux;
    int vet[TAM] = {9, 6, 5, 8, -4, 1};
    for (i = 0; i < TAM/2; i++){
        /// Troca entre os elementos
        aux = vet[i];
        vet[i] = vet[TAM-1-i];
        vet[TAM-1-i] = aux;
    }
    for (i = 0; i < TAM; i++)
        printf("%d ", vet[i]);
    return 0;
}
```

Management

Projects | Symbols | Files | FSymbols

Workspace
- Aula
  - Sources
    - main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 0 |
| vet | |
| [0] | 1 |
| [1] | 6 |
| [2] | 5 |
| [3] | 8 |
| [4] | -4 |
| [5] | 9 |
| aux | 9 |

Fim da inversão

main.c

```c
#include <stdio.h>
#define TAM 6
int main(){
    int i, aux;
    int vet[TAM] = {9, 6, 5, 8, -4, 1};
    for (i = 0; i < TAM/2; i++){
        /// Troca entre os elementos
        aux = vet[i];
        vet[i] = vet[TAM-1-i];
        vet[TAM-1-i] = aux;
    }
    for (i = 0; i < TAM; i++)
        printf("%d ", vet[i]);
    return 0;
}
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 1 |
| vet | |
| [0] | 1 |
| [1] | 6 |
| [2] | 5 |
| [3] | 8 |
| [4] | -4 |
| [5] | 9 |
| aux | 9 |

**Incremento da variável e teste da condição <u>verdadeiro</u>.**

```
4   int i, aux;
5   int vet[TAM] = {9, 6, 5, 8, -4, 1};
6   for (i = 0; i < TAM/2; i++){
7       /// Troca entre os elementos
8       aux = vet[i];
9       vet[i] = vet[TAM-1-i];
10      vet[TAM-1-i] = aux;
11  }
12  for (i = 0; i < TAM; i++)
13      printf("%d ", vet[i]);
14  return 0;
15 }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 1 |
| vet | |
| [0] | 1 |
| [1] | 6 |
| [2] | 5 |
| [3] | 8 |
| [4] | -4 |
| [5] | 9 |
| aux | 6 |

**Início da inversão do segundo elemento com o penúltimo**

main.c

```c
1  #include <stdio.h>
2  #define TAM 6
   t main(){
       int i, aux;
       int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 1 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 5 |
| [3] | 8 |
| [4] | -4 |
| [5] | 9 |
| aux | 6 |

Continuação...

main.c

```
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 1 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 5 |
| [3] | 8 |
| [4] | 6 |
| [5] | 9 |
| aux | 6 |

Fim da inversão

main.c

```
1   #include <stdio.h>
2   #define TAM 6
3   int main(){
4       int i, aux;
5       int vet[TAM] = {9, 6, 5, 8, -4, 1};
6       for (i = 0; i < TAM/2; i++){
7           /// Troca entre os elementos
8           aux = vet[i];
9           vet[i] = vet[TAM-1-i];
10          vet[TAM-1-i] = aux;
11      }
12      for (i = 0; i < TAM; i++)
13          printf("%d ", vet[i]);
14      return 0;
15  }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 2 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 5 |
| [3] | 8 |
| [4] | 6 |
| [5] | 9 |
| aux | 6 |

**Incremento da variável e teste da condição <u>verdadeiro</u>.**

```
4   int i, aux;
5   int vet[TAM] = {9, 6, 5, 8, -4, 1};
6   for (i = 0; i < TAM/2; i++){
7       /// Troca entre os elementos
8       aux = vet[i];
9       vet[i] = vet[TAM-1-i];
10      vet[TAM-1-i] = aux;
11  }
12  for (i = 0; i < TAM; i++)
13      printf("%d ", vet[i]);
14  return 0;
15 }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 2 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 5 |
| [3] | 8 |
| [4] | 6 |
| [5] | 9 |
| aux | 5 |

**Início da inversão do terceiro elemento com o antepenúltimo**

main.c

```c
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 2 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 8 |
| [4] | 6 |
| [5] | 9 |
| aux | 5 |

Continuação...

```
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 2 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |
| aux | 5 |

Fim da inversão

```
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 3 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |
| aux | 5 |

**Incremento da variável e teste da condição <u>falso</u>.**

```
#include <stdio.h>
#define TAM 6
int main(){
    int i, aux;
    int vet[TAM] = {9, 6, 5, 8, -4, 1};
    for (i = 0; i < TAM/2; i++){
        /// Troca entre os elementos
        aux = vet[i];
        vet[i] = vet[TAM-1-i];
        vet[TAM-1-i] = aux;
    }
    for (i = 0; i < TAM; i++)
        printf("%d ", vet[i]);
    return 0;
}
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 0 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |
| aux | 5 |

Variável inicializada novamente

Impressões dos elementos do vetor

```c
#include <stdio.h>
#define TAM 6
int main(){
    int i, aux;
    int vet[TAM] = {9, 6, 5, 8, -4, 1};
    for (i = 0; i < TAM/2; i++){
        /// Troca entre os elementos
        aux = vet[i];
        vet[i] = vet[TAM-1-i];
        vet[TAM-1-i] = aux;
    }
    for (i = 0; i < TAM; i++)
        printf("%d ", vet[i]);
    return 0;
}
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 0 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |
| aux | 5 |

main.c

```c
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

Impressões dos elementos do vetor

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

```
1
```

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 1 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |
| aux | 5 |

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

main.c

```c
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

```
1
```

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 1 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |
| aux | 5 |

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

main.c

```c
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

```
1 -4
```

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 2 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |
| aux | 5 |

main.c

```c
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

```
1 -4
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 2 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |
| aux | 5 |

main.c

```c
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

```
1 -4 8
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 3 |
| aux | 5 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |

Management

Projects Symbols Files FSymbols

- Workspace
  - Aula
    - Sources
      - main.c

main.c

```c
#include <stdio.h>
#define TAM 6
int main(){
    int i, aux;
    int vet[TAM] = {9, 6, 5, 8, -4, 1};
    for (i = 0; i < TAM/2; i++){
        /// Troca entre os elementos
        aux = vet[i];
        vet[i] = vet[TAM-1-i];
        vet[TAM-1-i] = aux;
    }
    for (i = 0; i < TAM; i++)
        printf("%d ", vet[i]);
    return 0;
}
```

```
1 -4 8
```

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 3 |
| aux | 5 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |

main.c

```c
#include <stdio.h>
#define TAM 6
int main(){
    int i, aux;
    int vet[TAM] = {9, 6, 5, 8, -4, 1};
    for (i = 0; i < TAM/2; i++){
        /// Troca entre os elementos
        aux = vet[i];
        vet[i] = vet[TAM-1-i];
        vet[TAM-1-i] = aux;
    }
    for (i = 0; i < TAM; i++)
        printf("%d ", vet[i]);
    return 0;
}
```

```
1 -4 8 5
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 4 |
| aux | 5 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |

Management

Projects Symbols Files FSymbols

- Workspace
  - Aula
    - Sources
      - main.c

main.c

```c
#include <stdio.h>
#define TAM 6
int main(){
    int i, aux;
    int vet[TAM] = {9, 6, 5, 8, -4, 1};
    for (i = 0; i < TAM/2; i++){
        /// Troca entre os elementos
        aux = vet[i];
        vet[i] = vet[TAM-1-i];
        vet[TAM-1-i] = aux;
    }
    for (i = 0; i < TAM; i++)
        printf("%d ", vet[i]);
    return 0;
}
```

```
1 -4 8 5
```

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 4 |
| aux | 5 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

main.c

```c
#include <stdio.h>
#define TAM 6
int main(){
    int i, aux;
    int vet[TAM] = {9, 6, 5, 8, -4, 1};
    for (i = 0; i < TAM/2; i++){
        /// Troca entre os elementos
        aux = vet[i];
        vet[i] = vet[TAM-1-i];
        vet[TAM-1-i] = aux;
    }
    for (i = 0; i < TAM; i++)
        printf("%d ", vet[i]);
    return 0;
}
```

```
1 -4 8 5 6
```

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 5 |
| aux | 5 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |

Management

Projects Symbols Files FSymbols

- Workspace
  - Aula
    - Sources
      - main.c

main.c

```c
#include <stdio.h>
#define TAM 6
int main(){
    int i, aux;
    int vet[TAM] = {9, 6, 5, 8, -4, 1};
    for (i = 0; i < TAM/2; i++){
        /// Troca entre os elementos
        aux = vet[i];
        vet[i] = vet[TAM-1-i];
        vet[TAM-1-i] = aux;
    }
    for (i = 0; i < TAM; i++)
        printf("%d ", vet[i]);
    return 0;
}
```

```
1 -4 8 5 6
```

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 5 |
| aux | 5 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |

Management

Projects Symbols Files FSymbols

- Workspace
  - Aula
    - Sources
      - main.c

main.c

```c
#include <stdio.h>
#define TAM 6
int main(){
    int i, aux;
    int vet[TAM] = {9, 6, 5, 8, -4, 1};
    for (i = 0; i < TAM/2; i++){
        /// Troca entre os elementos
        aux = vet[i];
        vet[i] = vet[TAM-1-i];
        vet[TAM-1-i] = aux;
    }
    for (i = 0; i < TAM; i++)
        printf("%d ", vet[i]);
    return 0;
}
```

```
1 -4 8 5 6 9
```

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| Function argument | |
|---|---|
| Locals | |
| i | 6 |
| aux | 5 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |

**Incremento da variável e teste da condição <u>falso</u>.**

```
3   int main(){
4       int i, aux;
5       int vet[TAM] = {9, 6, 5, 8, -4, 1};
6       for (i = 0; i < TAM/2; i++){
7           /// Troca entre os elementos
8           aux = vet[i];
9           vet[i] = vet[TAM-1-i];
10          vet[TAM-1-i] = aux;
11      }
12      for (i = 0; i < TAM; i++)
13          printf("%d ", vet[i]);
14      return 0;
15  }
16
```

```
1 -4 8 5 6 9
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Execução

main.c [Aula] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Watches (new)

| | |
|---|---|
| Function argument | |
| Locals | |
| i | 6 |
| aux | 5 |
| vet | |
| [0] | 1 |
| [1] | -4 |
| [2] | 8 |
| [3] | 5 |
| [4] | 6 |
| [5] | 9 |

main.c

```c
1  #include <stdio.h>
2  #define TAM 6
3  int main(){
4      int i, aux;
5      int vet[TAM] = {9, 6, 5, 8, -4, 1};
6      for (i = 0; i < TAM/2; i++){
7          /// Troca entre os elementos
8          aux = vet[i];
9          vet[i] = vet[TAM-1-i];
10         vet[TAM-1-i] = aux;
11     }
12     for (i = 0; i < TAM; i++)
13         printf("%d ", vet[i]);
14     return 0;
15 }
16
```

FIM

```
1 -4 8 5 6 9
```

Management

Projects Symbols Files FSymbols

Workspace
Aula
Sources
main.c

# Dica

- **Para maiores informações sobre o debugger do Code::Blocks, acesse o endereço abaixo:**

**http://wiki.codeblocks.org/index.php/Debugging_with_Code::Blocks**